

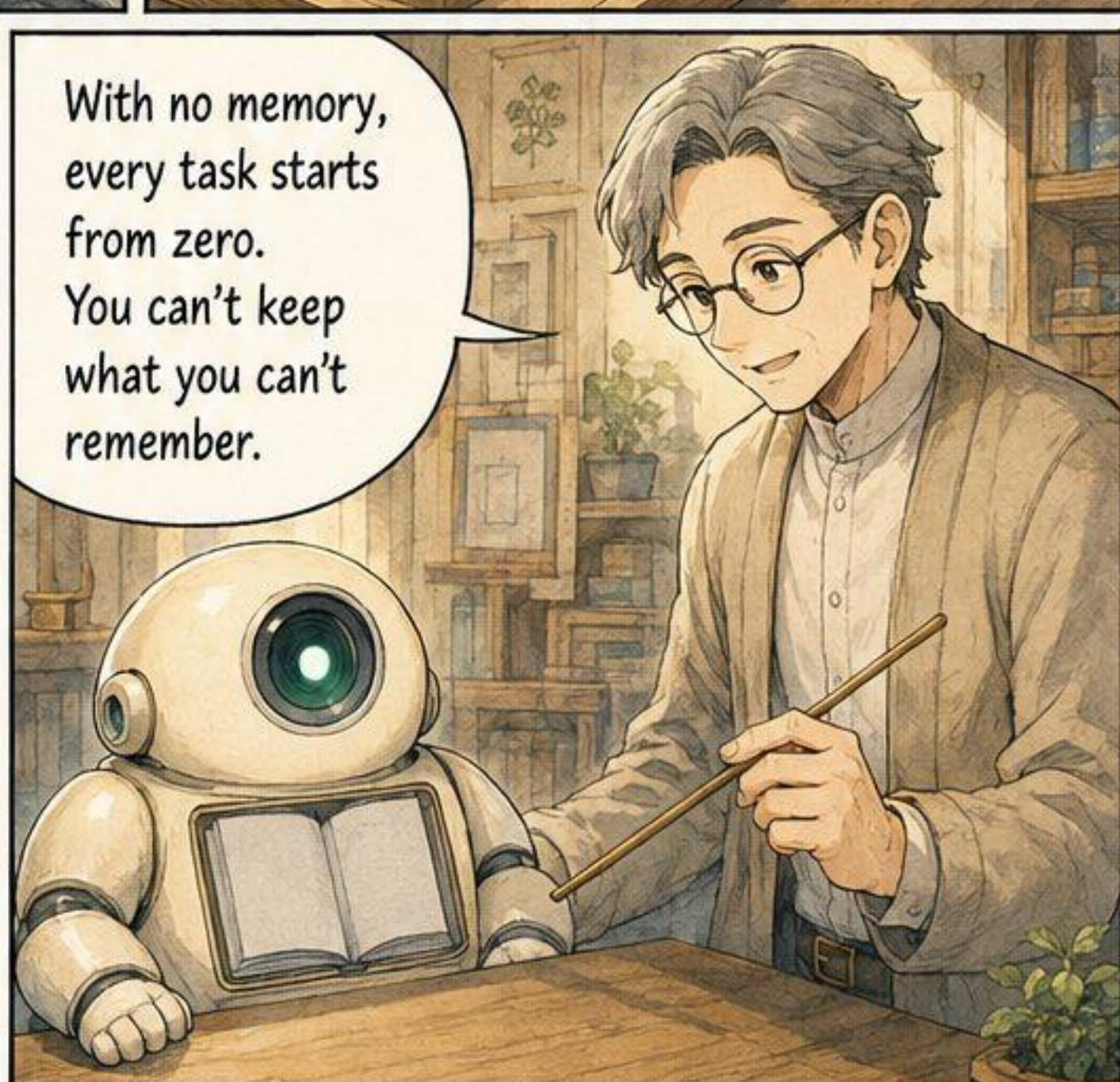
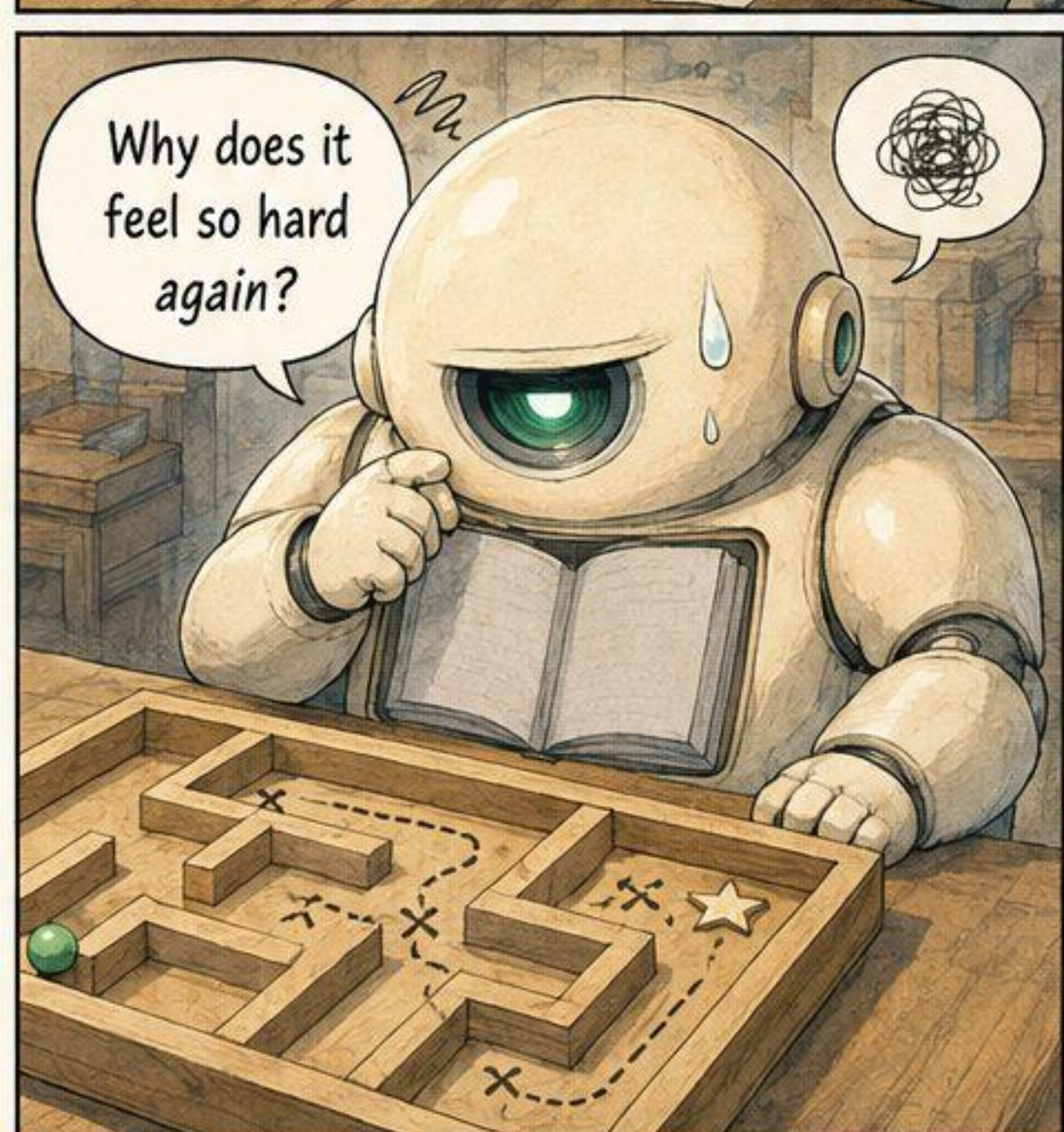
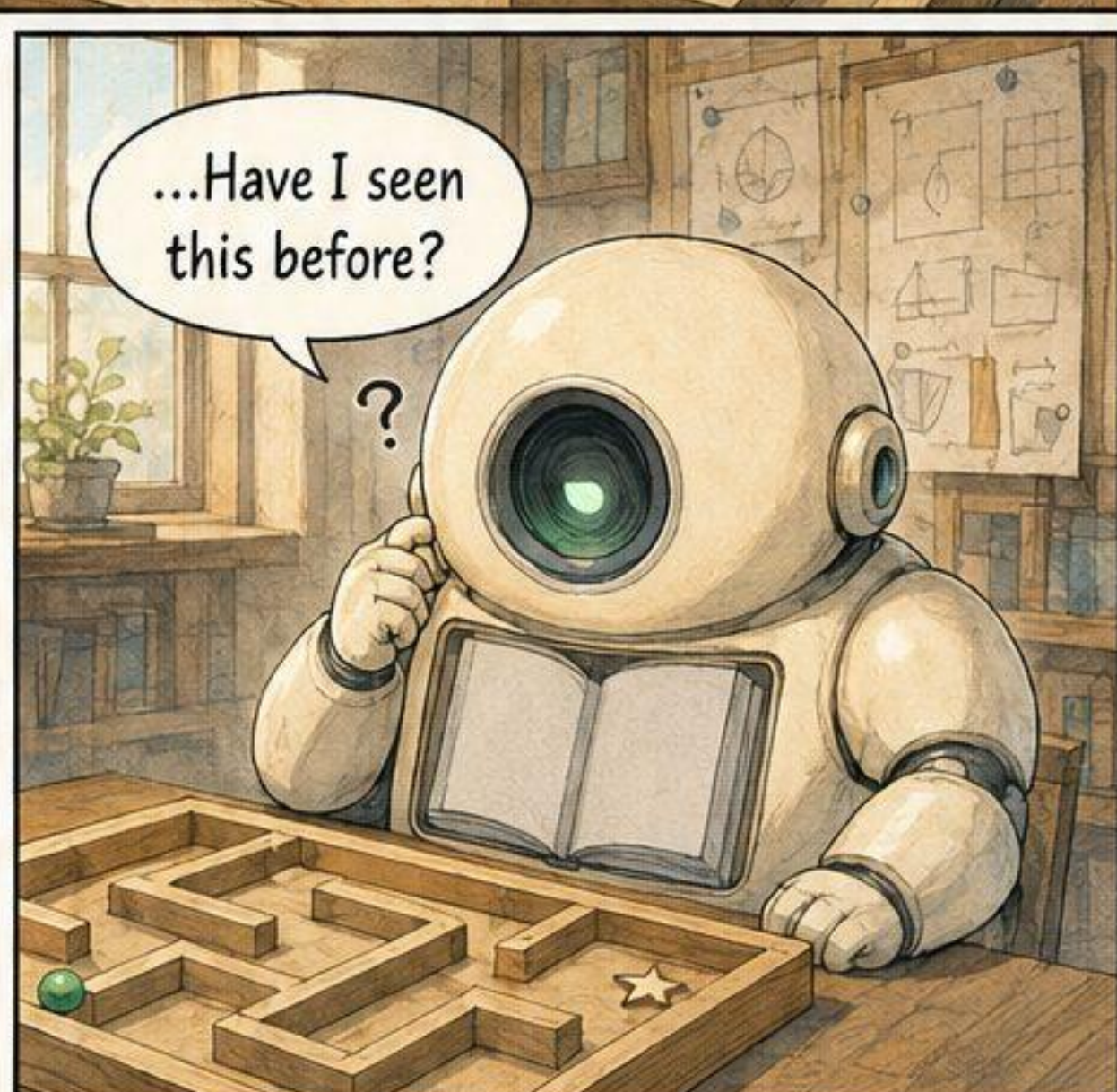
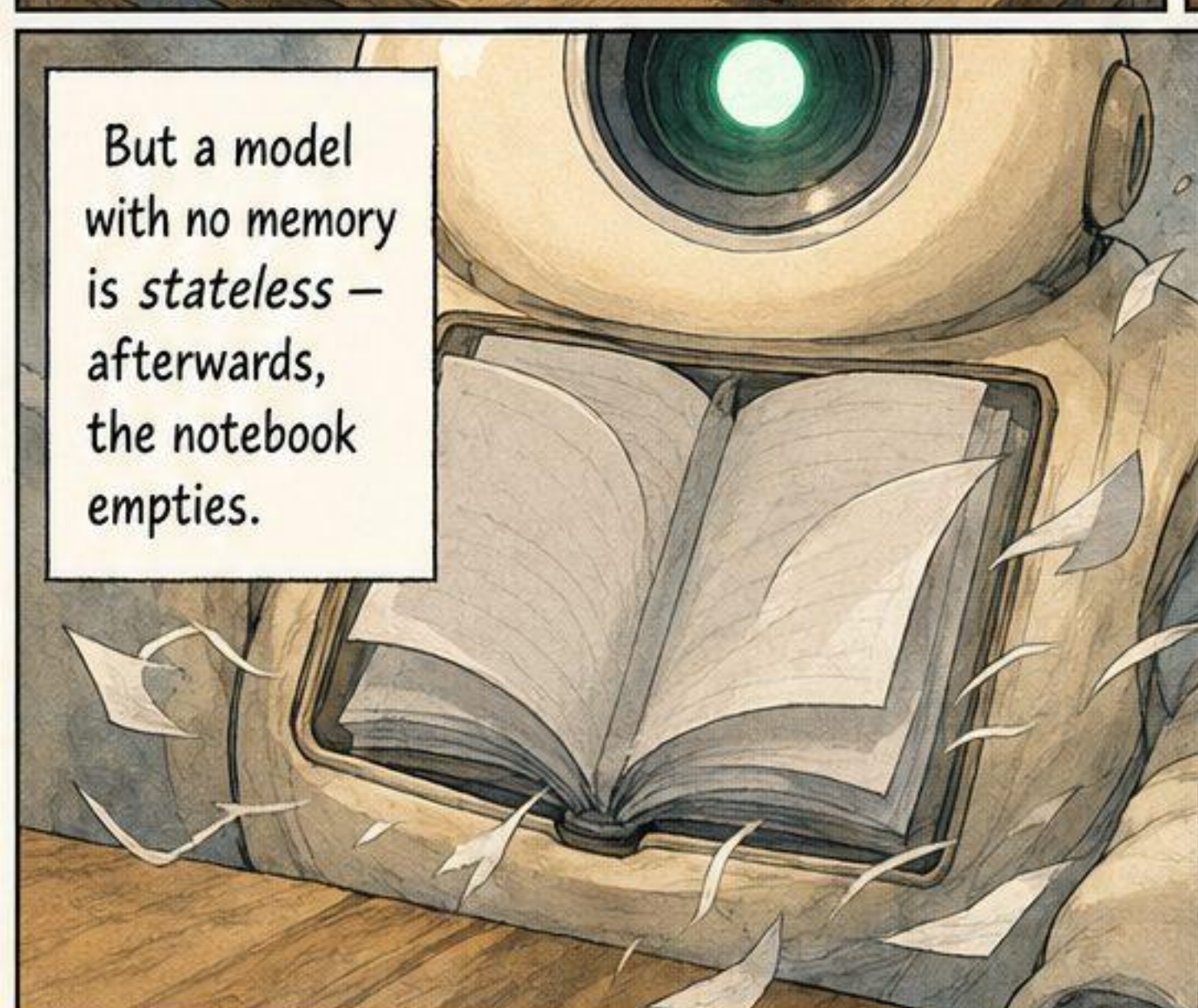
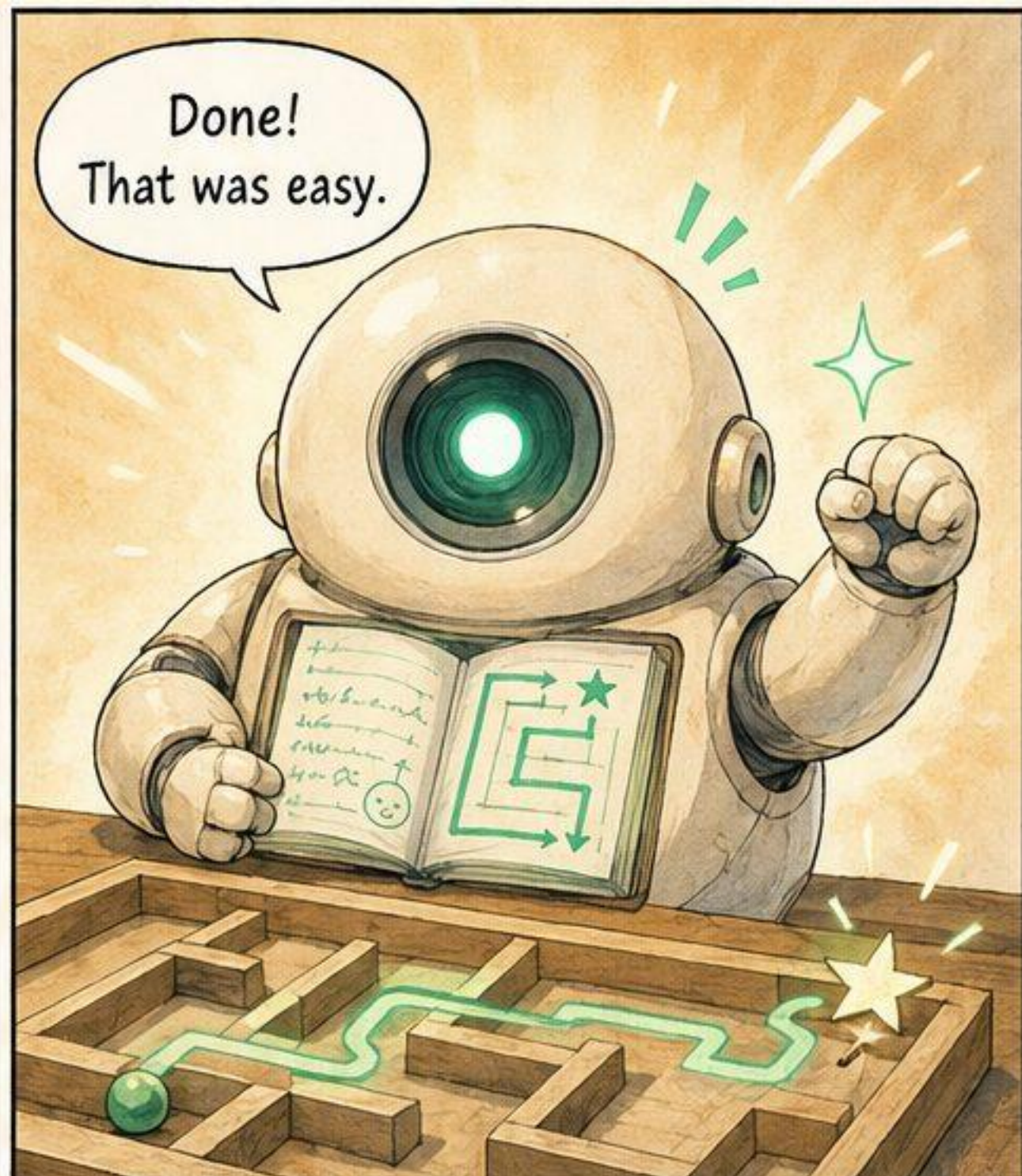
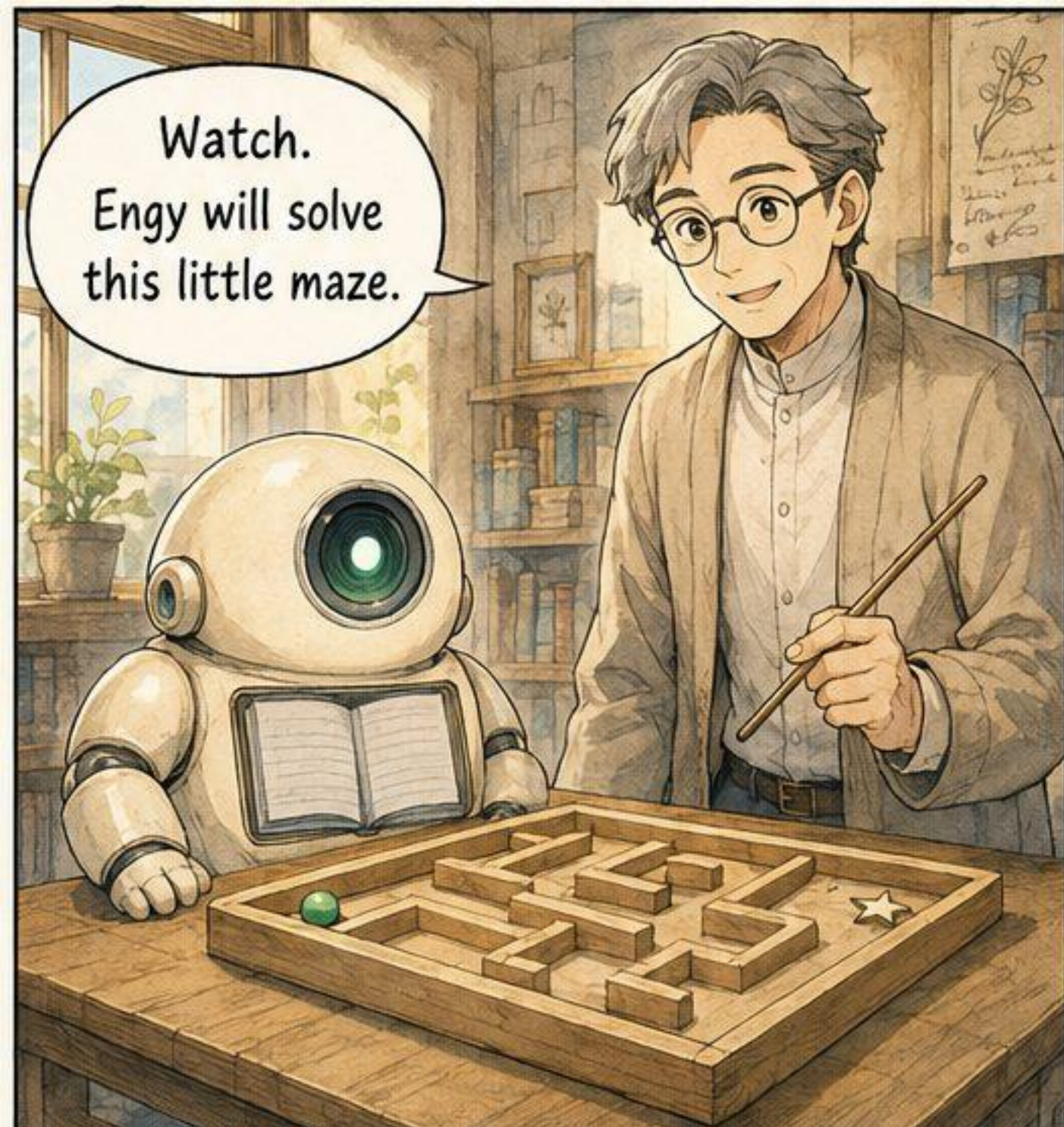
ENGRAM//KIT

educational manga reading pack

Two Papers, Two Manga

ALMA // AMA-Bench

github.com/romyilano/engram-kit



So we give Engy a memory module — a place to keep and reuse what it learns.

MEMORY MODULE

Now the note survives.
Stored experience
can be reused.

Maze rule:
Follow the open
path. Look ahead.

Wait —
I've done
something
like this
before...

Maze rule:
Follow the
open path.

General idea:
Find the path
that leads
to the goal.

Faster
this time!
Memory
helps.

It does.
But look closely
at the box itself...

Who designed these slots —
what to keep, where to file it,
what to forget?

?

MAZE

PUZZLES

PATHS

RULES

FACTS

OBSERVATIONS

TRICKS

OTHER

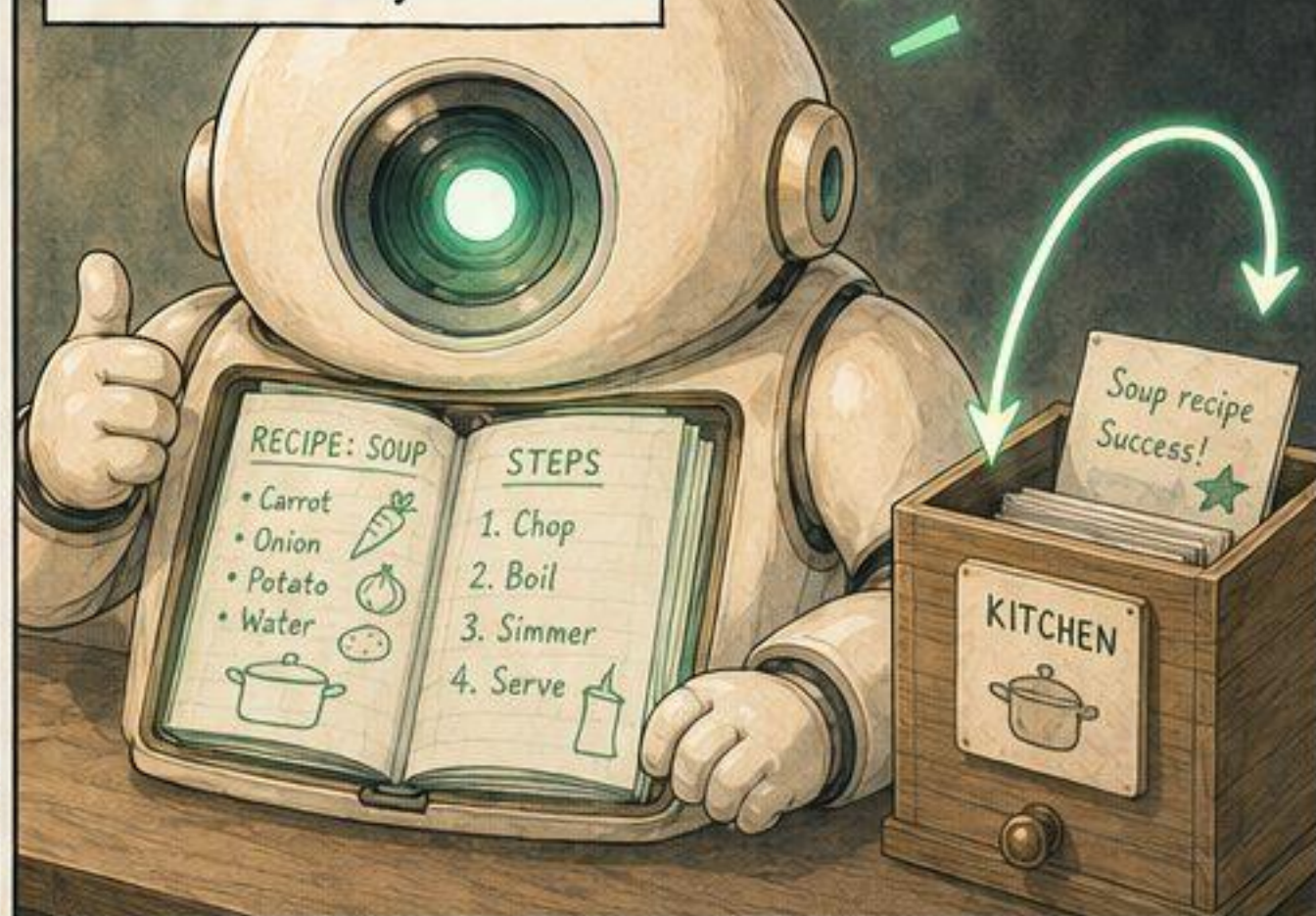
FIXED STRUCTURE

For years, we drew the memory design by hand – what to store, how to fetch it, when to update.

MEMORY DESIGN (HAND-CRAFTED)



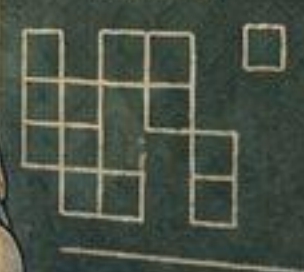
Hand-crafted for one task, it fits beautifully.



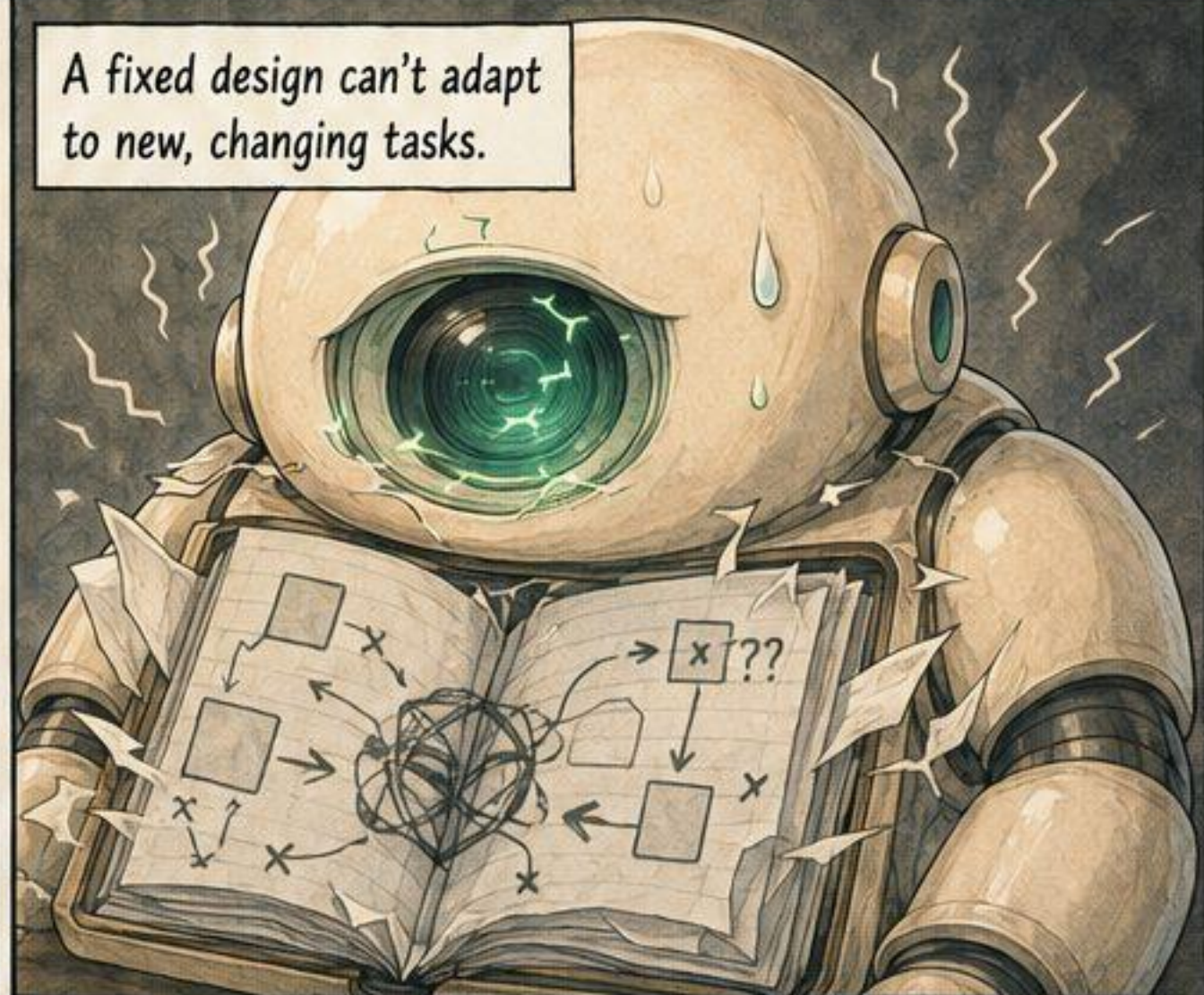
This world is... different. Nothing lines up.

SAME DESIGN APPLIED HERE

TILE PUZZLE WORLD



A fixed design can't adapt to new, changing tasks.



Diverse, shifting tasks each want a different memory. One hand-drawn shape can't serve them all.

MANY WORLDS



ONE FIXED DESIGN

STORE → RETRIEVE → UPDATE



So... what if the design didn't have to be drawn by hand?



KITCHEN WORLD



WORD PUZZLE WORLD



TILE GAME WORLD

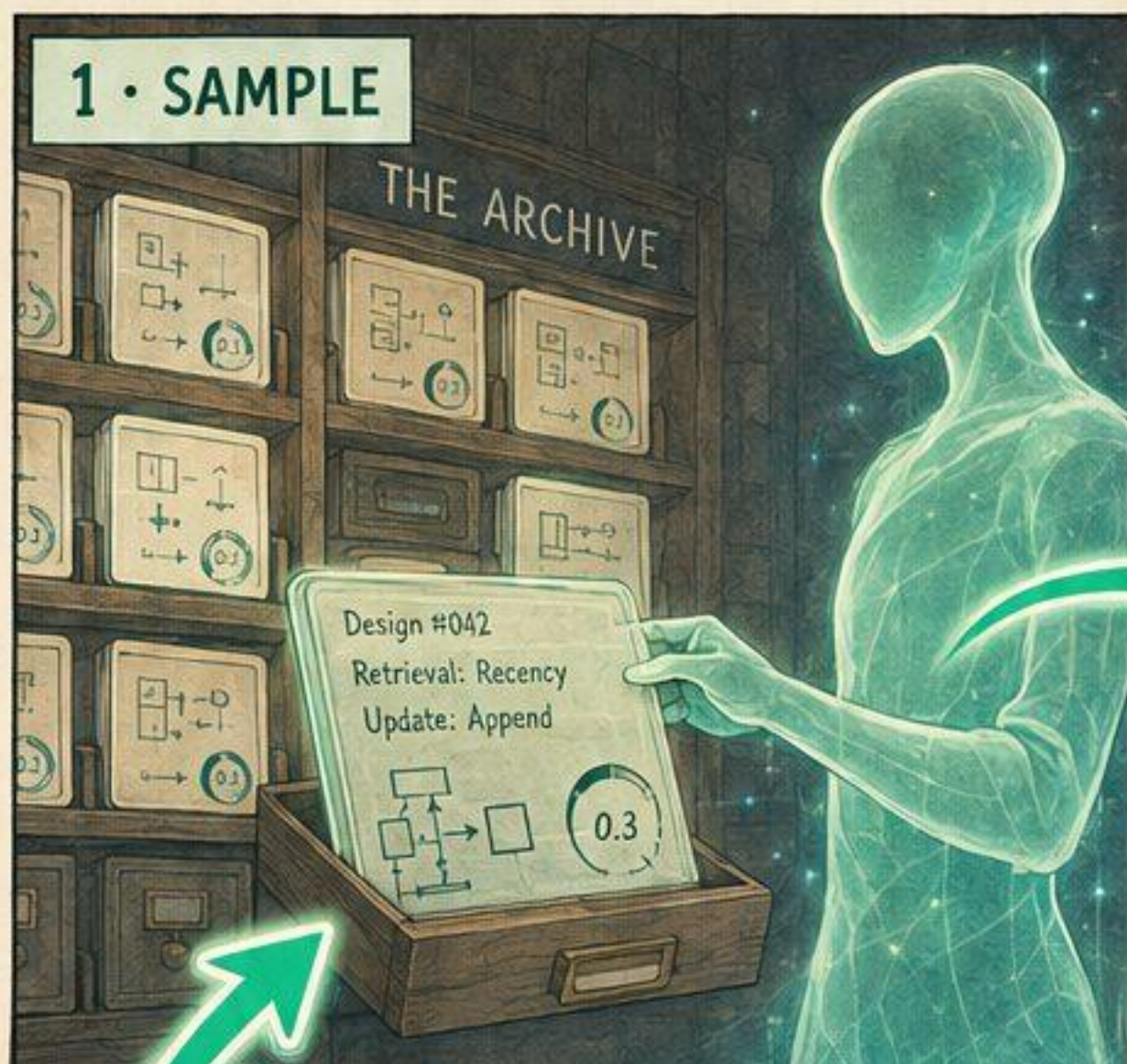


DUNGEON WORLD

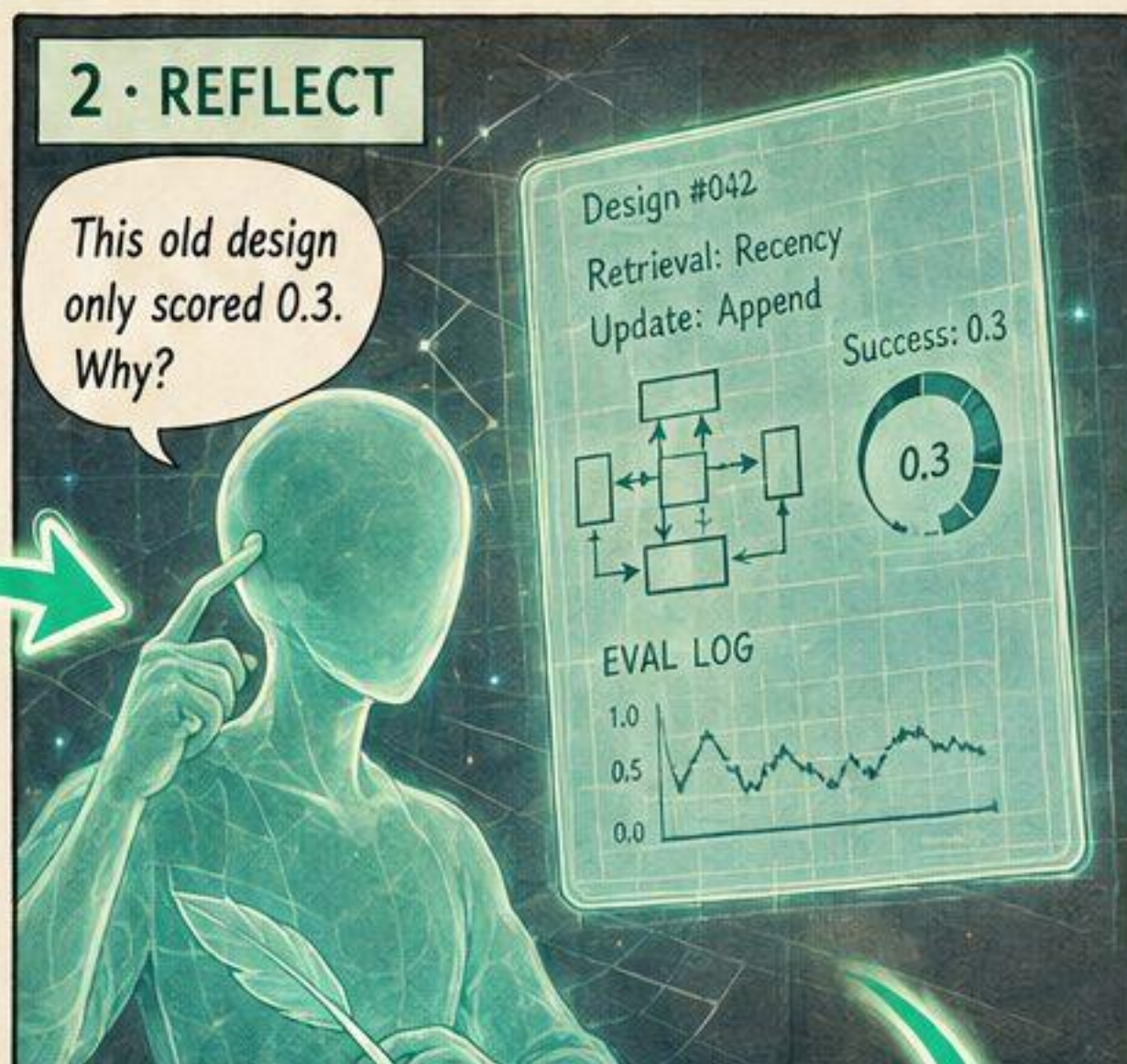
DIFFERENT WORLDS. DIFFERENT NEEDS. DIFFERENT MEMORY SHAPES.

ALMA doesn't do the task. It designs the memory — over and over.

1 • SAMPLE



2 • REFLECT

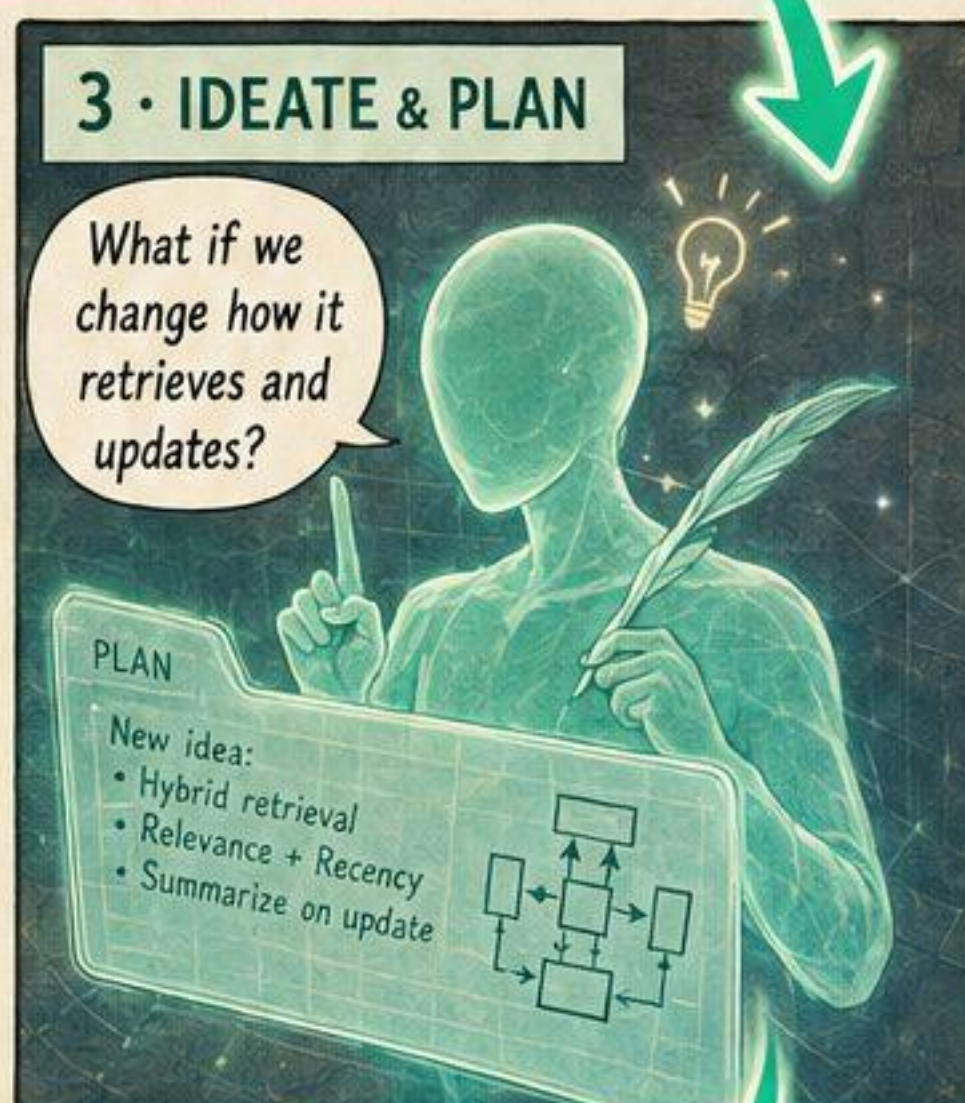


7 • STORE → repeat

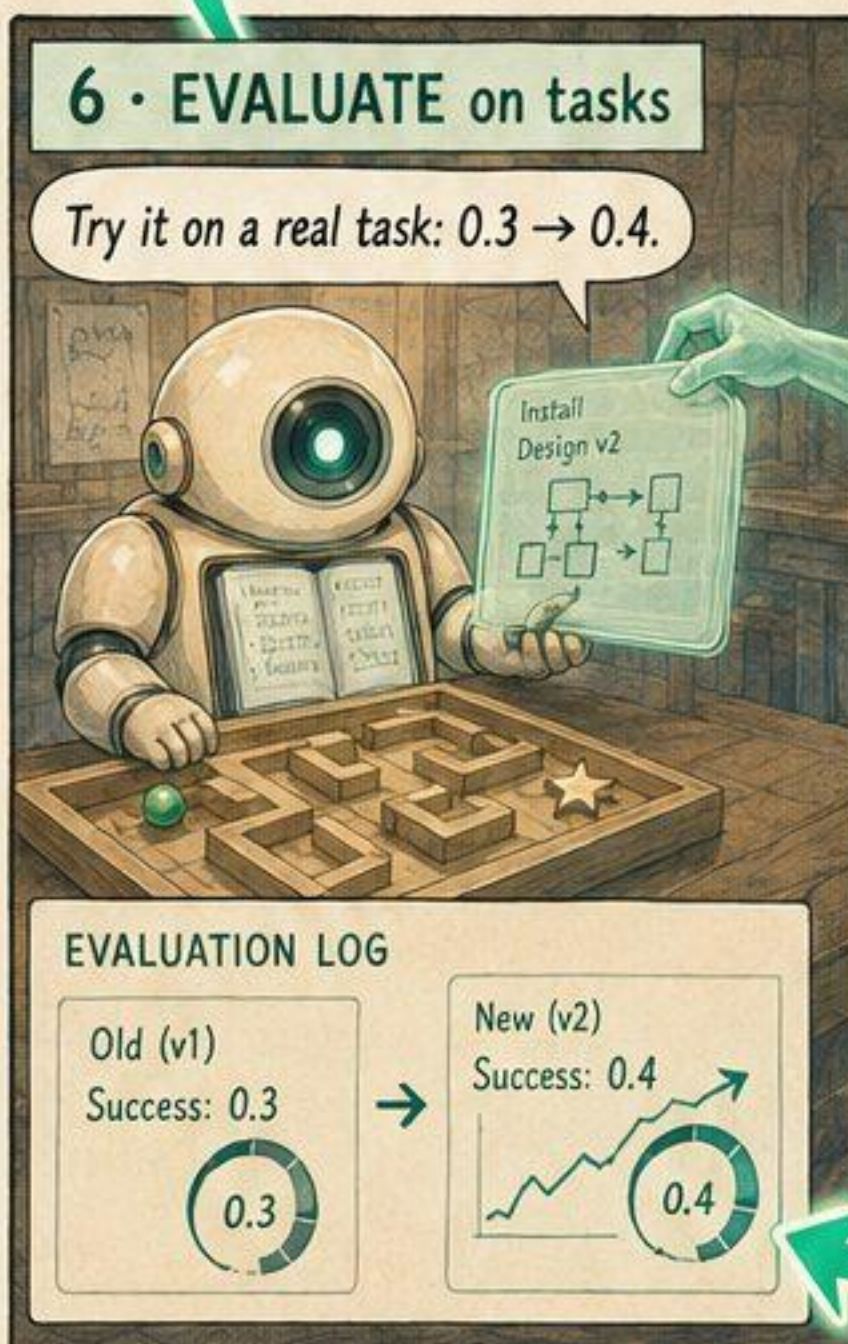


ALMA's
Meta Agent
meta-learns the
memory design
through an
open-ended loop.

3 • IDEATE & PLAN



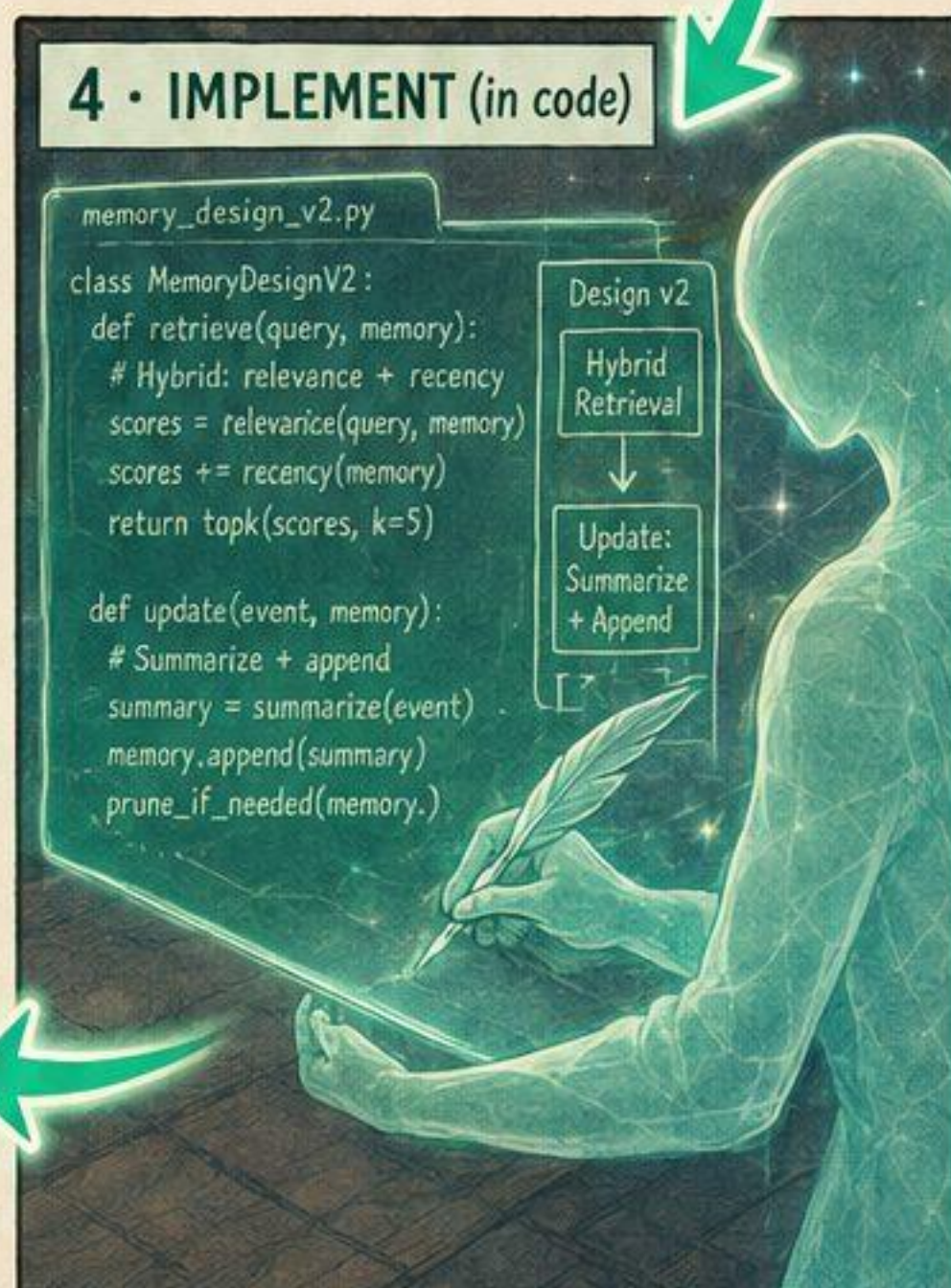
6 • EVALUATE on tasks



5 • VERIFY



4 • IMPLEMENT (in code)



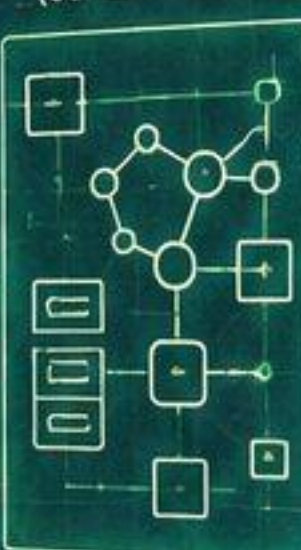
A memory design is just... code.

```
class MemoryDesign:
    Schema = {
        entities: [Note, Link, Fact],
        fields: {
            Note: [id, text, tags, time],
            Link: [src, dst, rel, time],
            Fact: [subj, pred, obj, conf, time]
        }
    }

    def retrieve(query, state):
        qv = embed(query)
        cand = graph_expand(state, qv, hops=2)
        scored = rank(cand, qv)
        return topk(scored, k=5)

    def update(state, obs):
        new = extract(obs)
        state = merge(state, new)
        state = decay(state)
        return state
```

MEMORY DESIGN (SCHEMATIC)



What we store, how we fetch it, how it changes.

(a) SCHEMA

the shape of what's stored

Note { id, text, tags, time }

Link { src, dst, rel, time }

Fact { subj, pred, obj, conf, time }

(b) RETRIEVAL

how a memory is fetched

Query

Embed

Graph Expand (hops=2)

Rank

Top-k Results

(c) UPDATE

how memory changes over time

New Observation

Extract

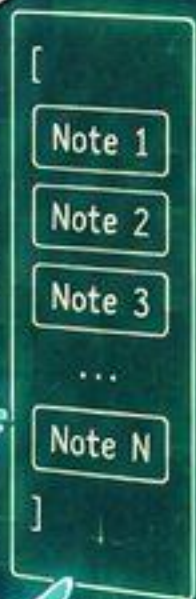
Merge

Decay / Prune

New State

A list? A database? A graph? Code can build any of them.

LIST DESIGN



DATABASE DESIGN

Notes			
id	text	tags	time
1	...	...	...
2	...	...	...

Links			
src	dst	rel	time
7	3	cites	...
...	...	...	...

Facts				
subj	pred	obj	conf	time
cat	likes	fish	0.9	...
...	...	...	...	...

GRAPH DESIGN



Different shapes. Different strengths. All expressible in code.

Install the new design straight into Engy's memory.

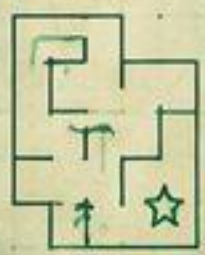
NEW DESIGN (Graph + Time Decay)



```
retrieve = graph_expand(...)
update = merge(...)
decay = time_weighted(...)
```

This shape suits this task!

TASK:



RETRIEVE

Query



Top-k

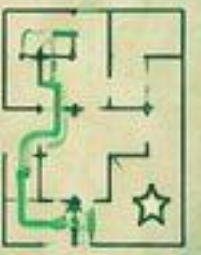
UPDATE

New Obs

Merge + Decay

New State

RESULT:



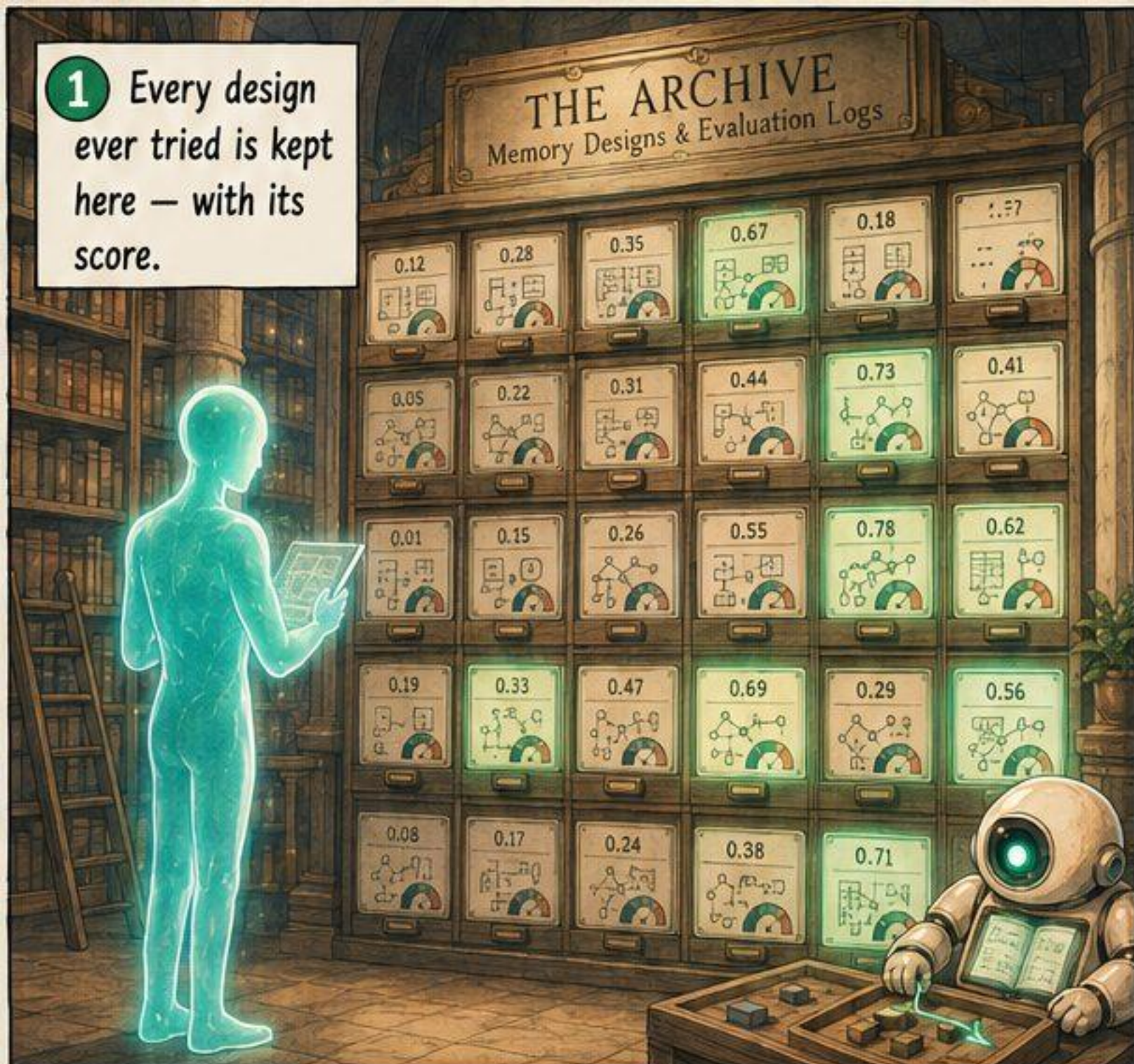
And the space of possible memories is endless.

```
class MemoryDesign:
    ...
```

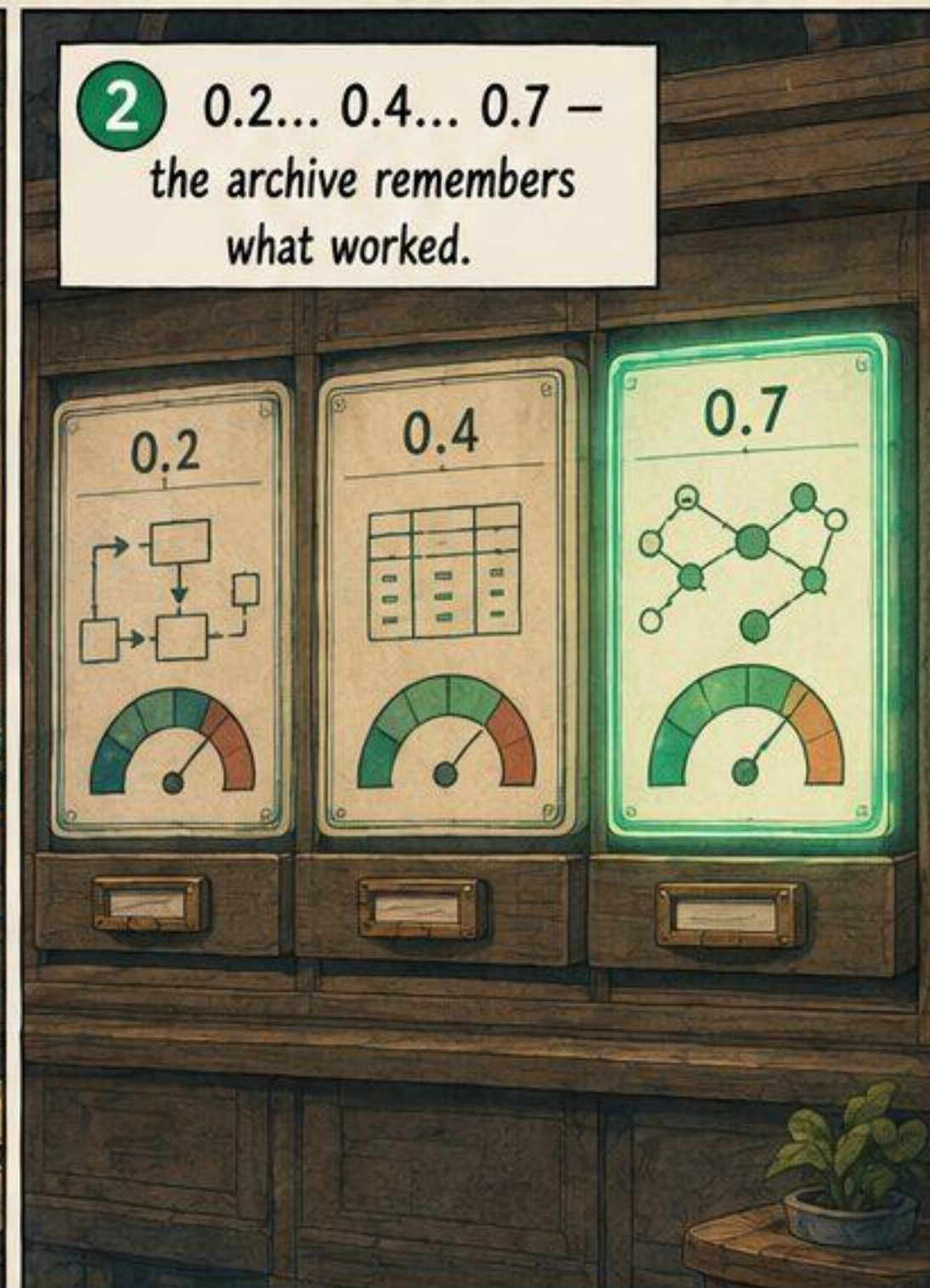
Countless designs. Infinite possibilities.



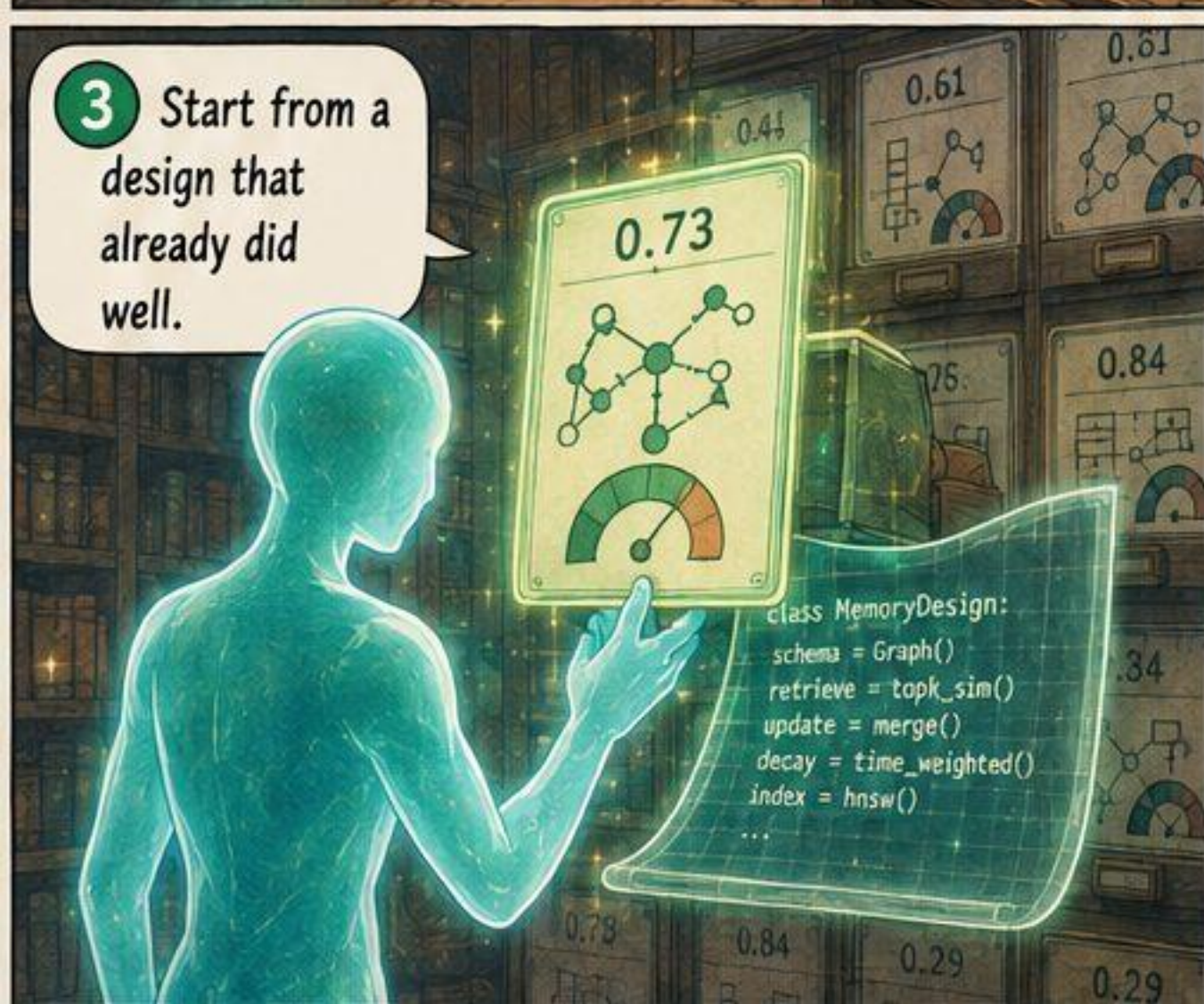
1 Every design ever tried is kept here — with its score.



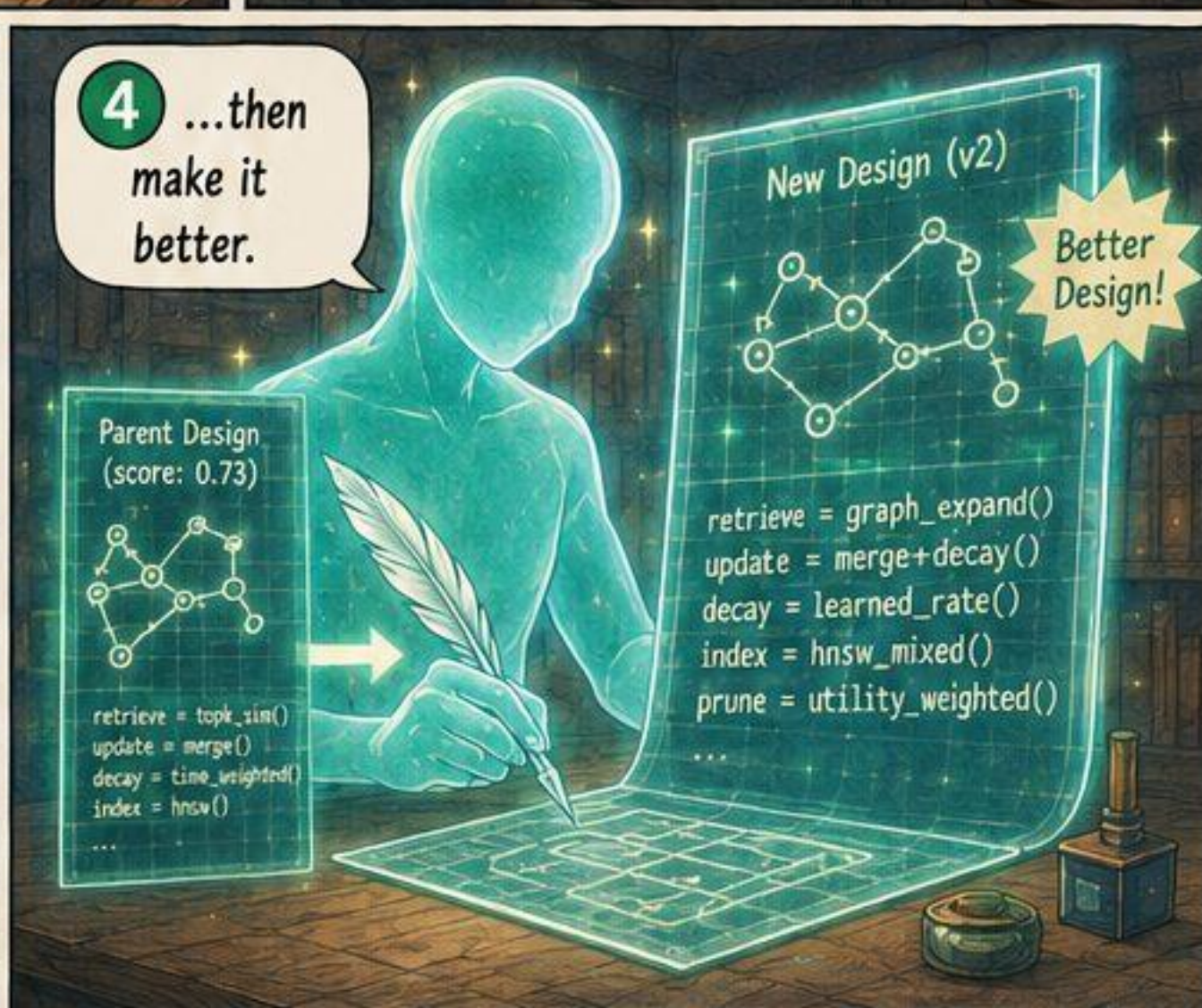
2 0.2... 0.4... 0.7 — the archive remembers what worked.



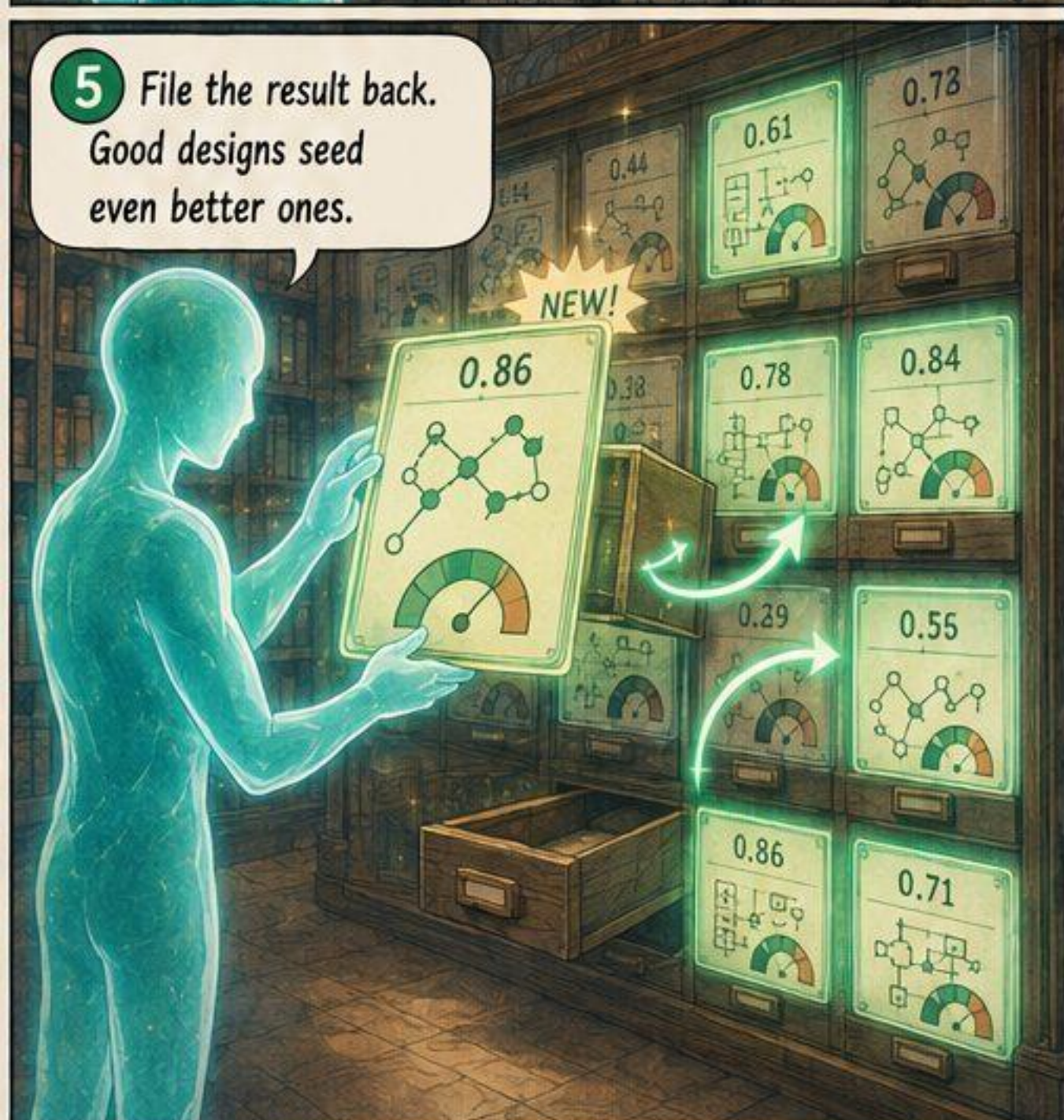
3 Start from a design that already did well.



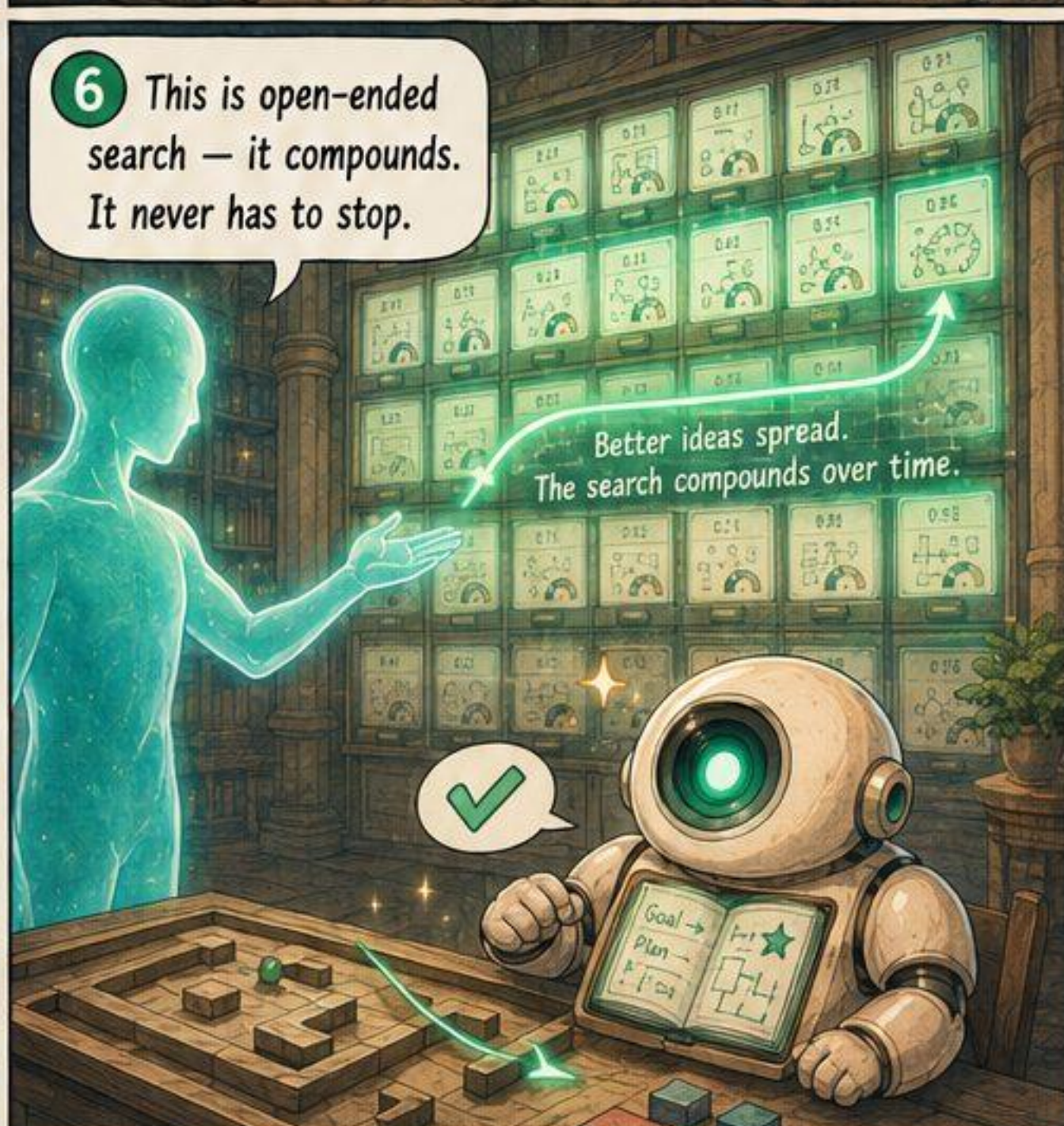
4 ...then make it better.



5 File the result back. Good designs seed even better ones.

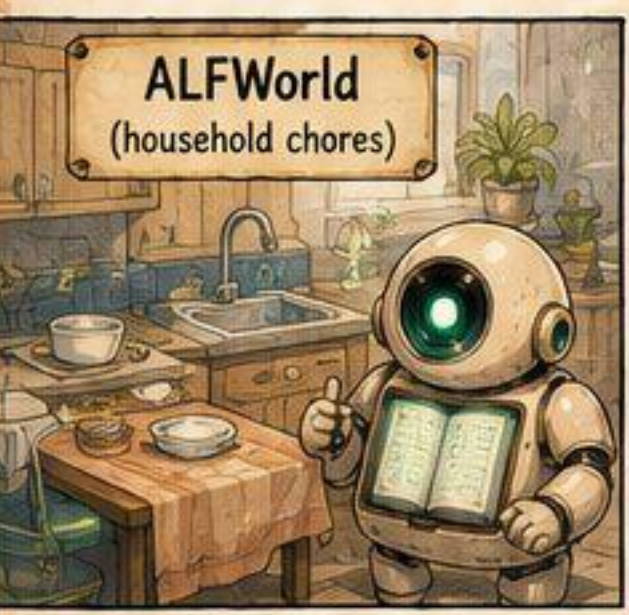


6 This is open-ended search — it compounds. It never has to stop.



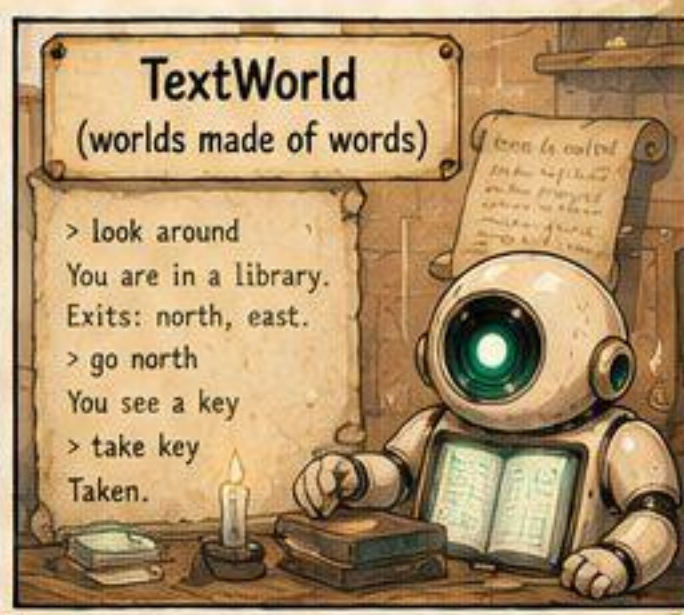
1 Four very different worlds:
ALFWorld · TextWorld · Baba Is AI · MiniHack.

ALFWorld
(household chores)



TextWorld
(worlds made of words)

> look around
You are in a library.
Exits: north, east.
> go north
You see a key
> take key
Taken.



Baba Is AI
(rule-pushing puzzle)

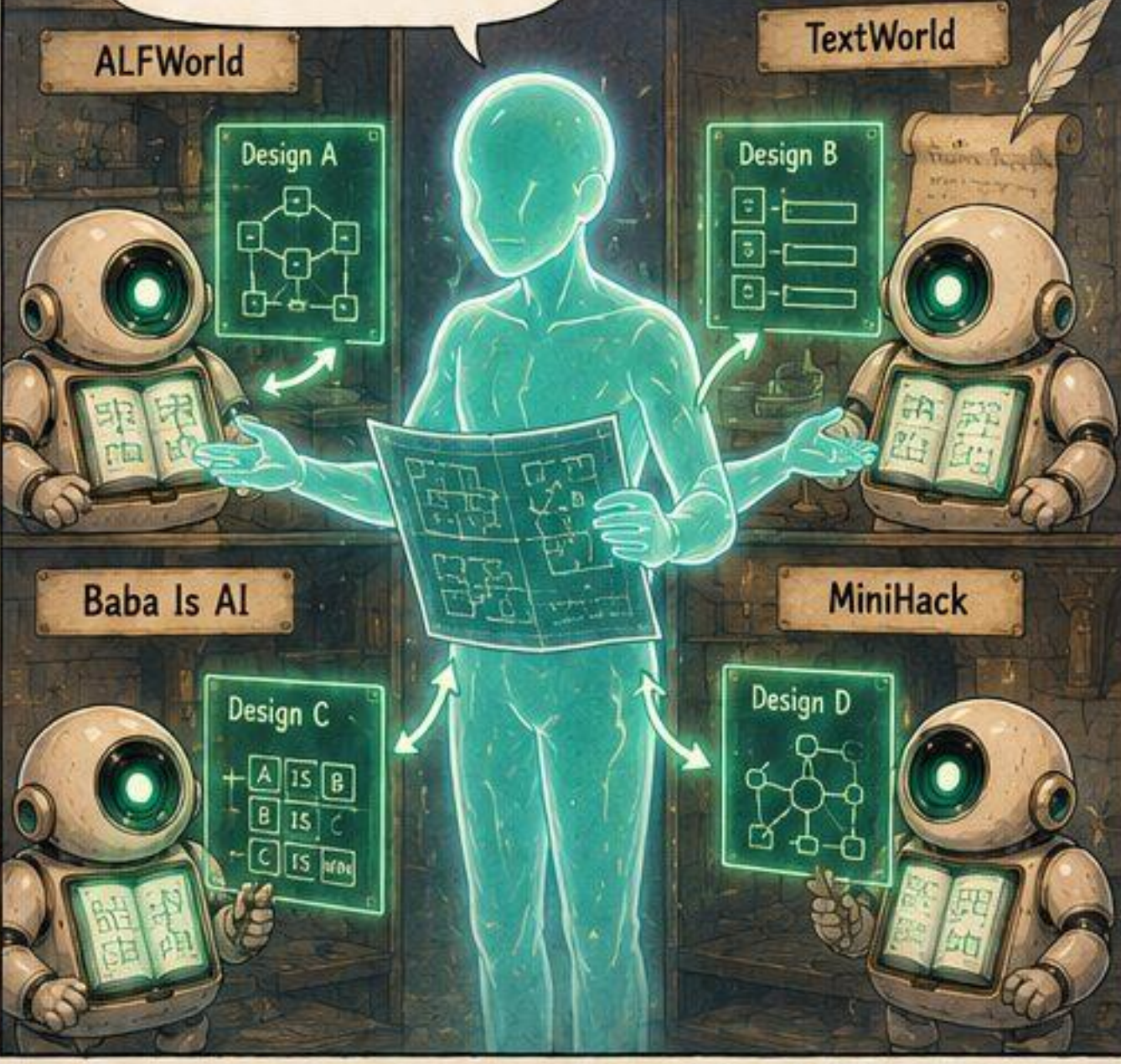


MiniHack
(dungeon crawl)

HP: 12/12
Gold: 23
Str: 8 Lev: 3



2 Each world,
its own memory design.



3 I'm actually
learning from
experience now!

TextWorld

> open door
You unlocked it.
> go north
You find a treasure chest!
You win!



Baba Is AI

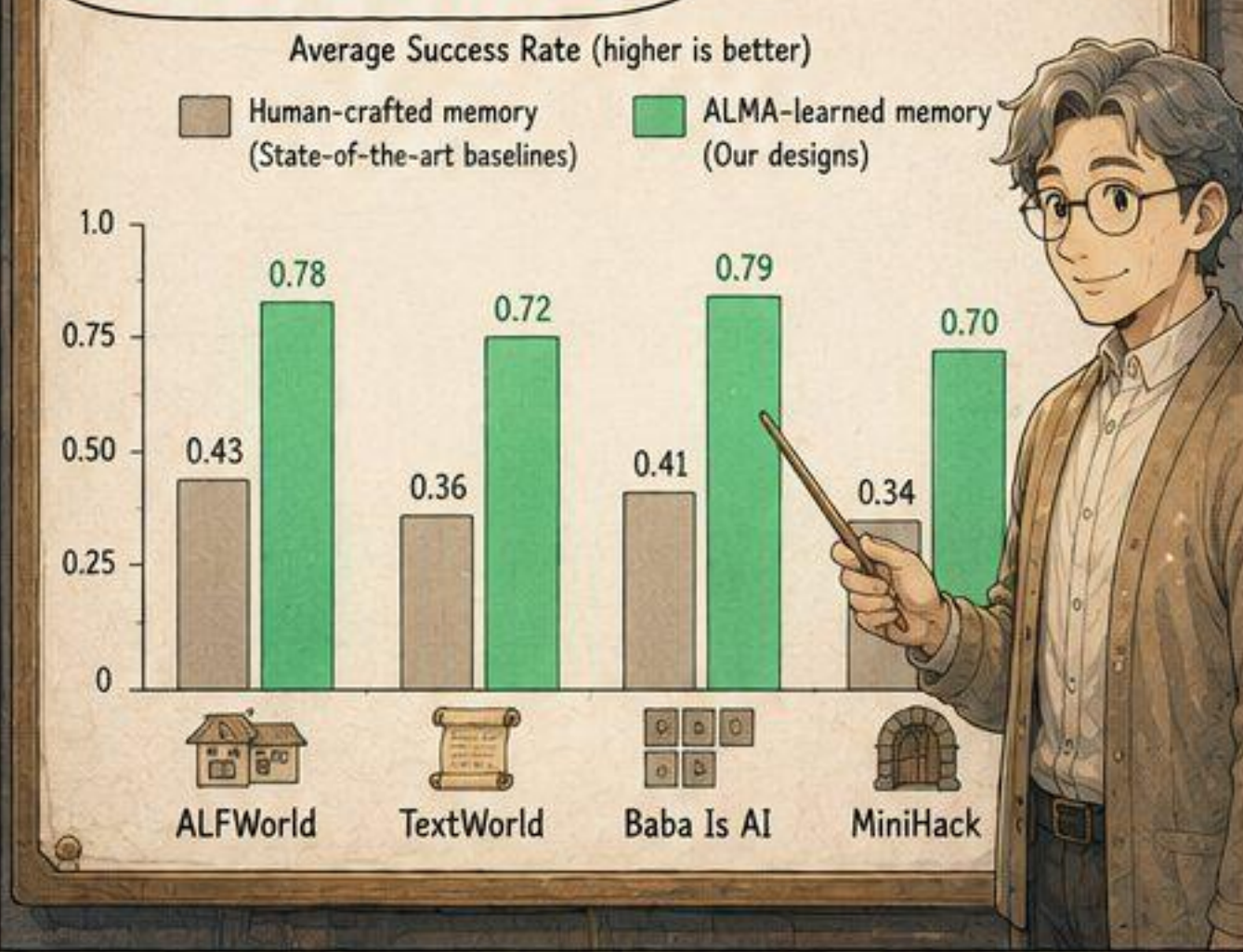


MiniHack

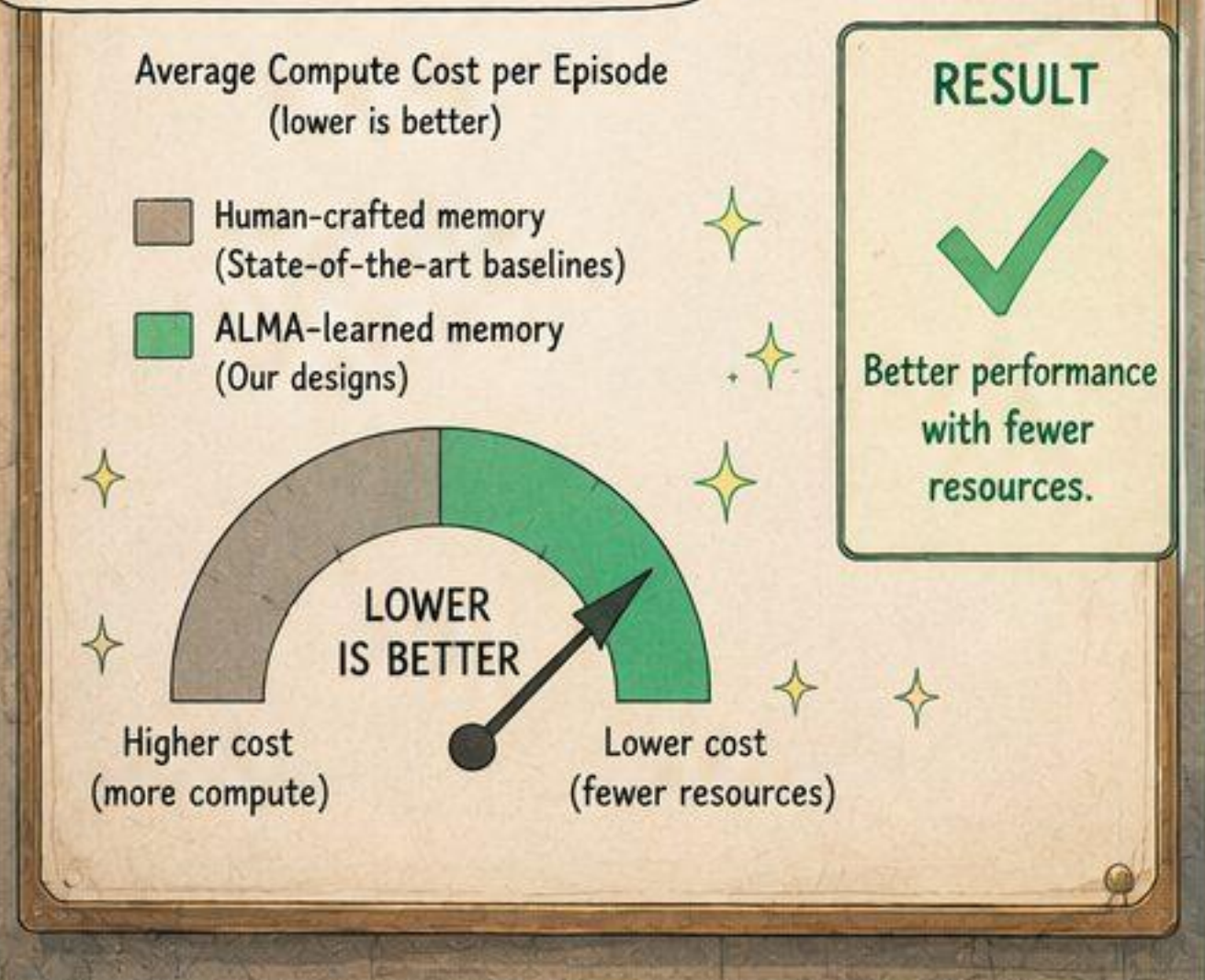
You hit the goblin.
Goblin defeated!
You find a potion.
Level up!
HP: 16/16 Lev: 4



4 Across every benchmark,
the learned designs beat
the hand-crafted ones.



5 And they cost less to run.

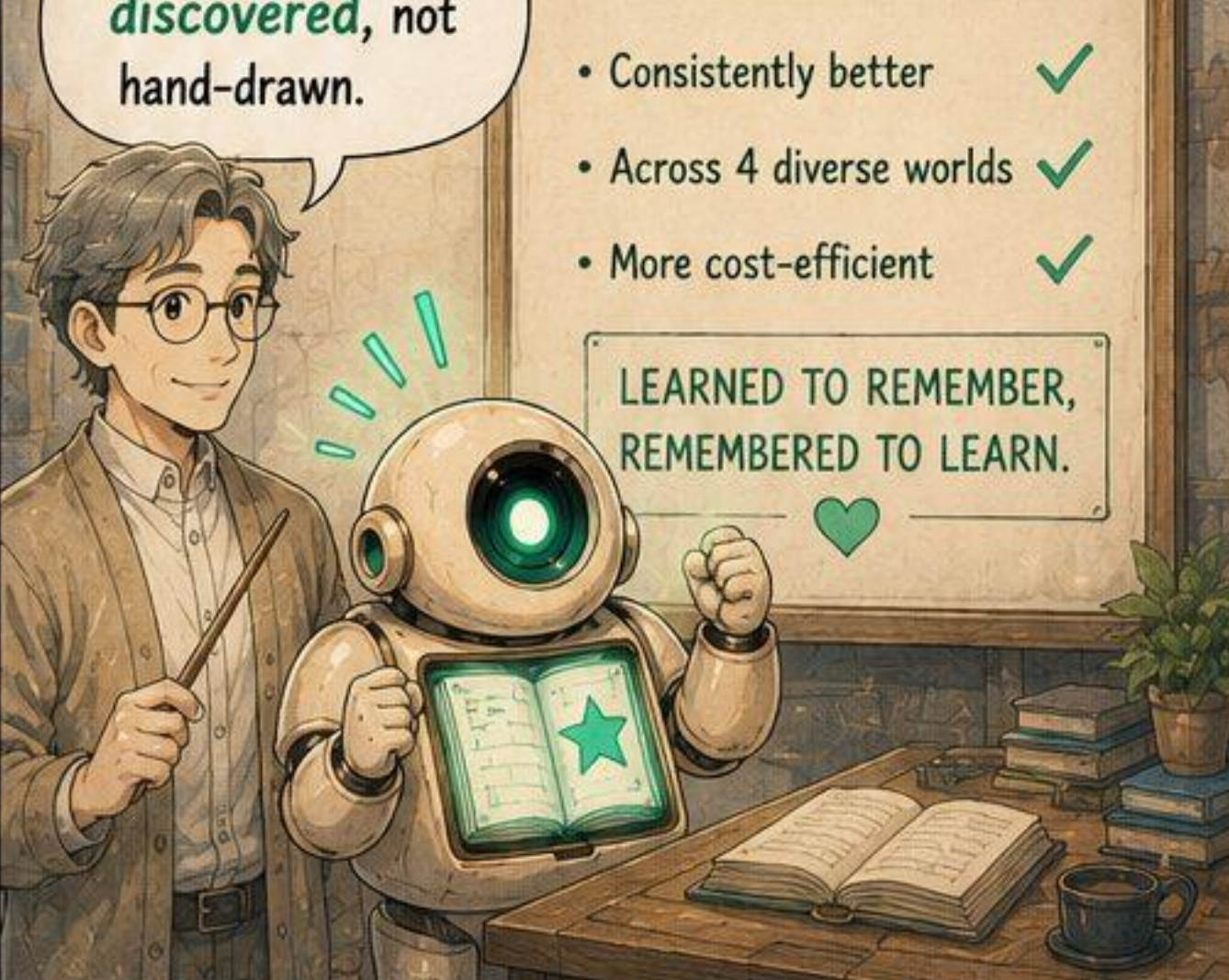


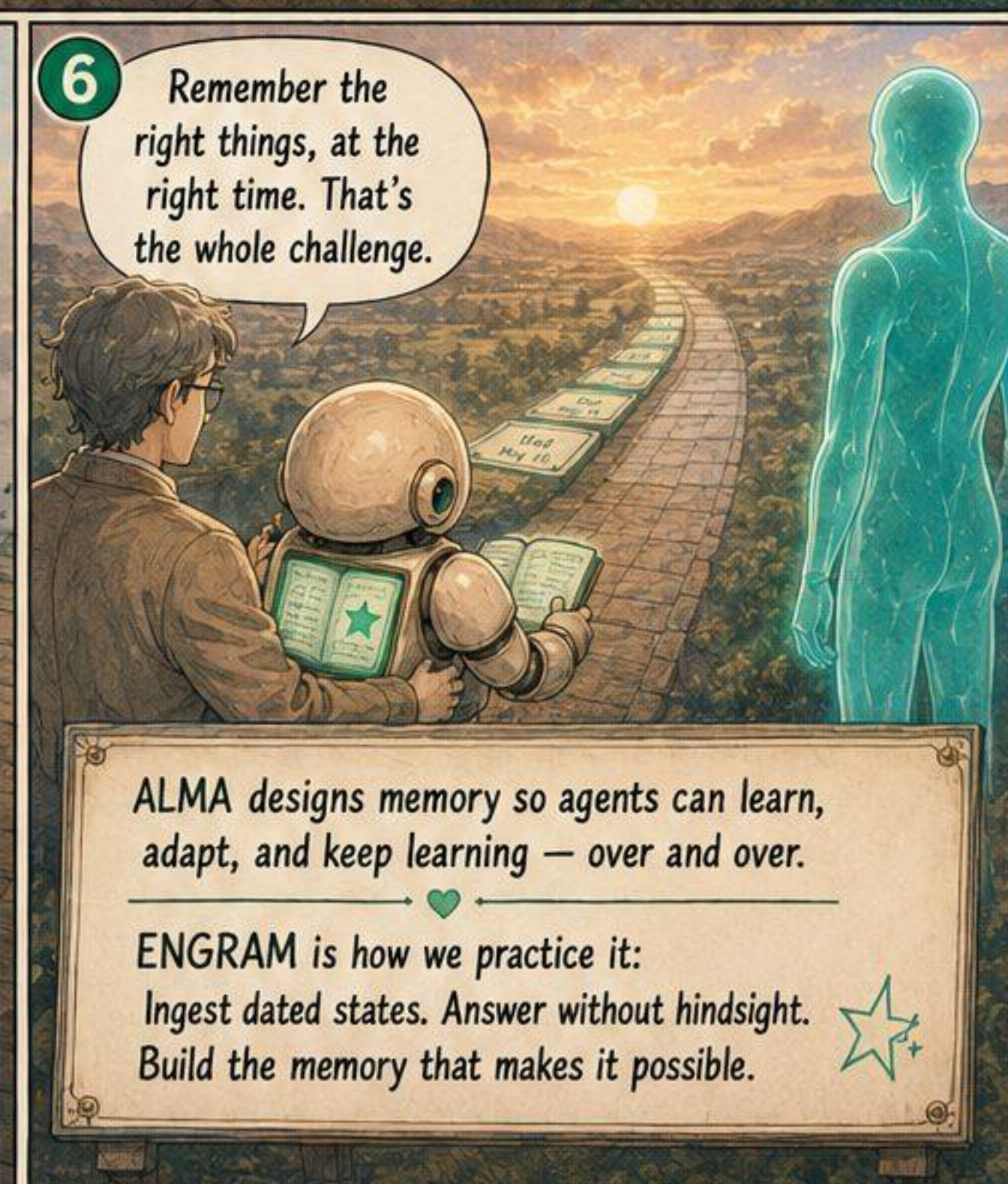
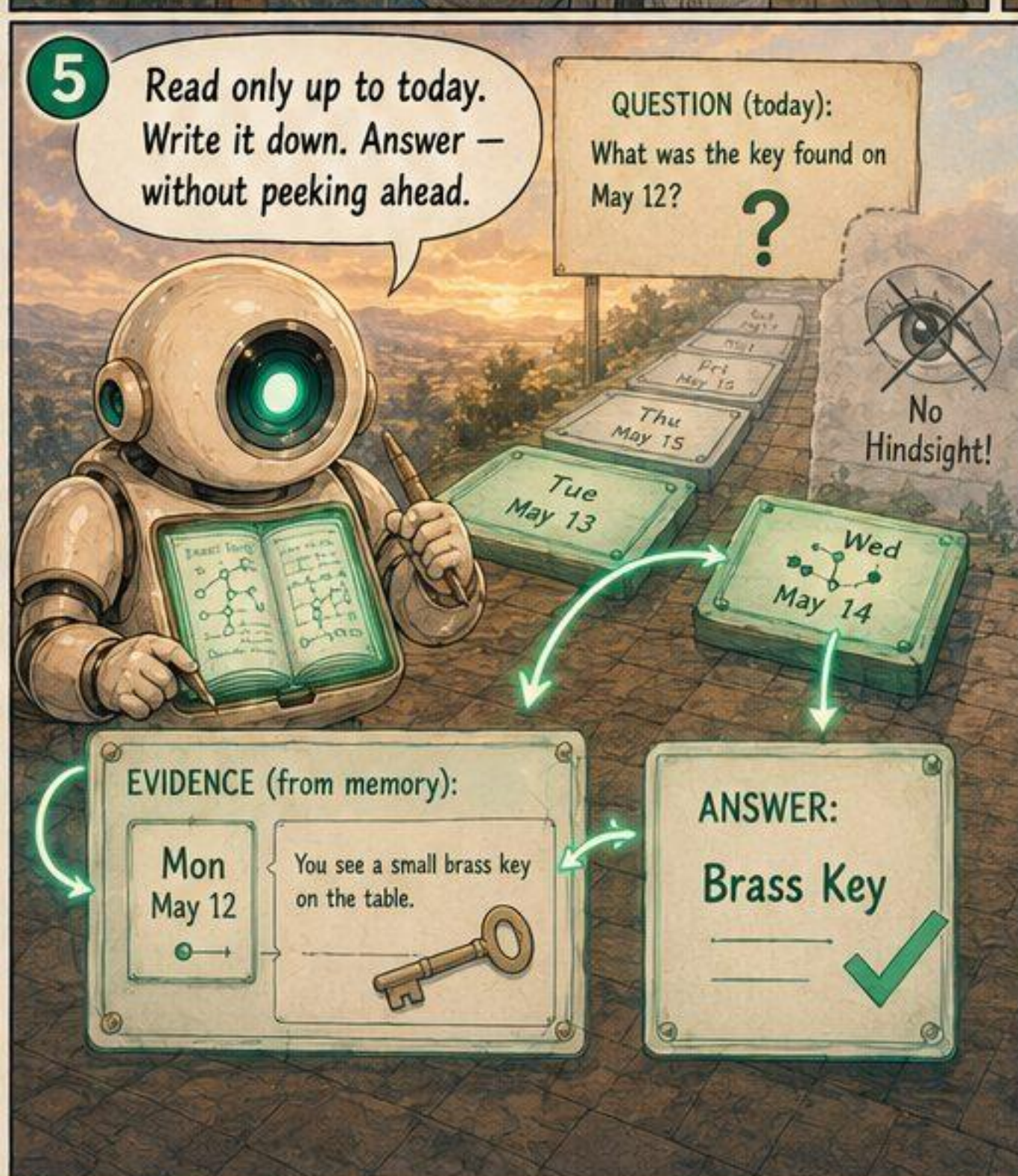
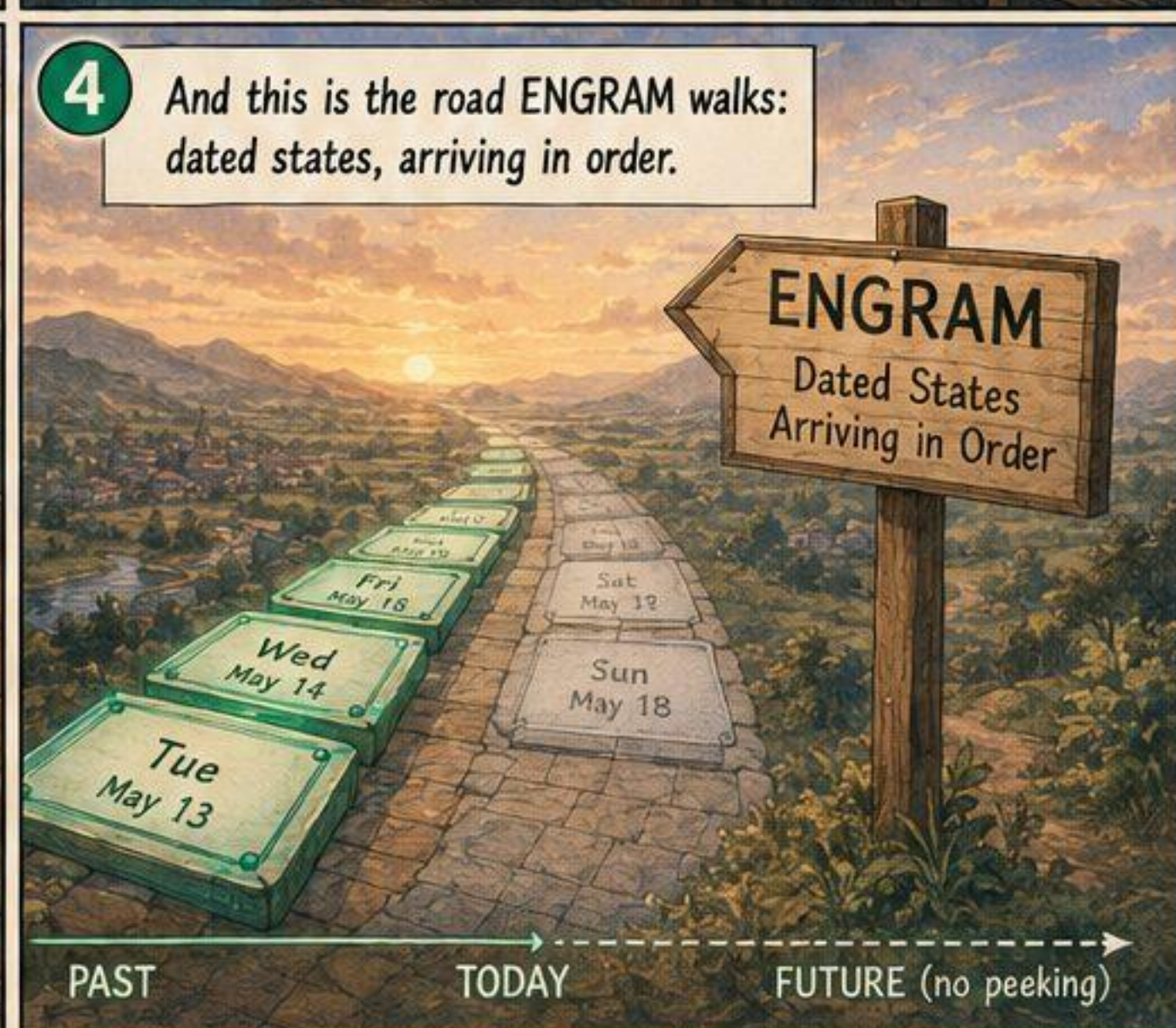
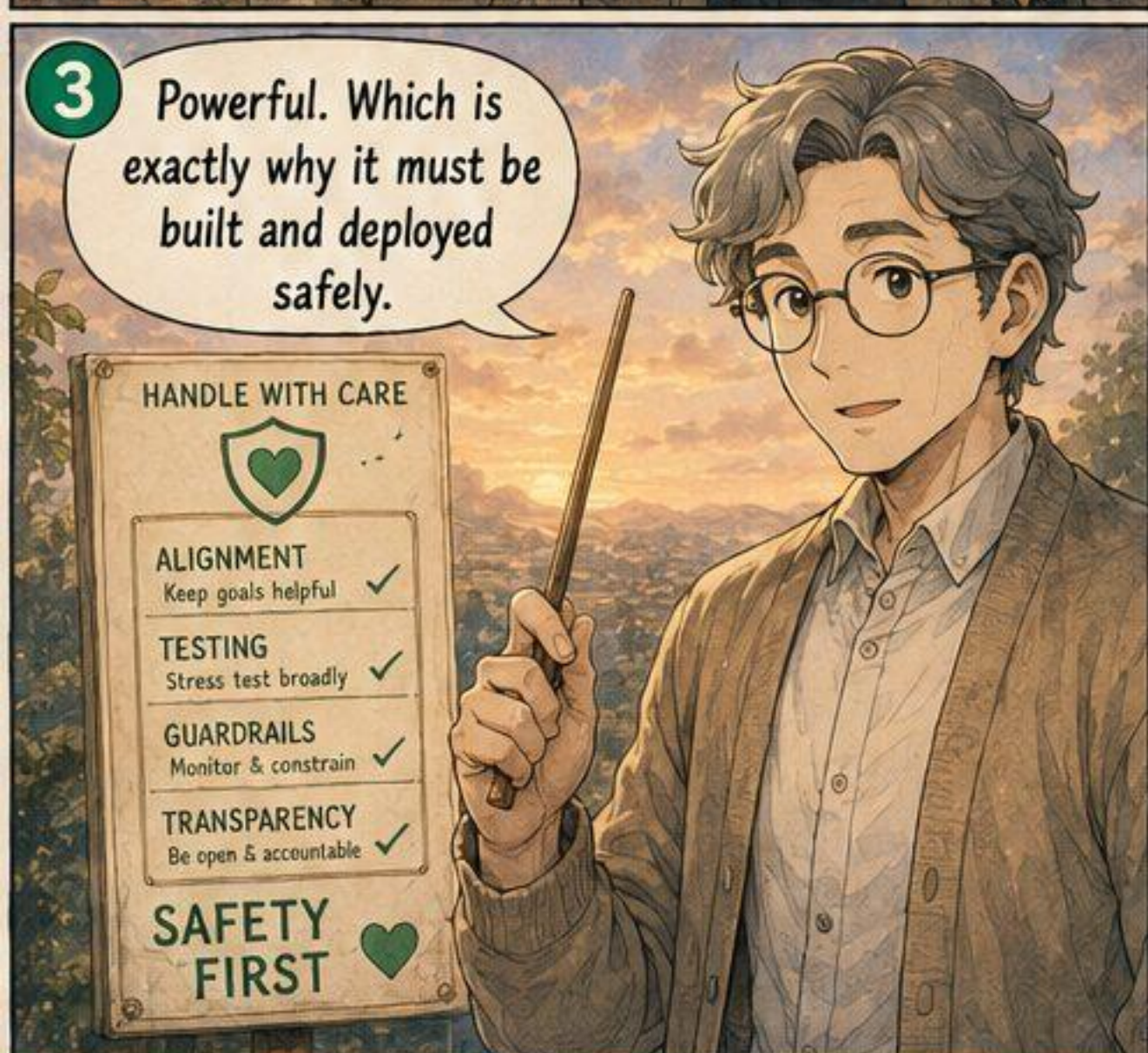
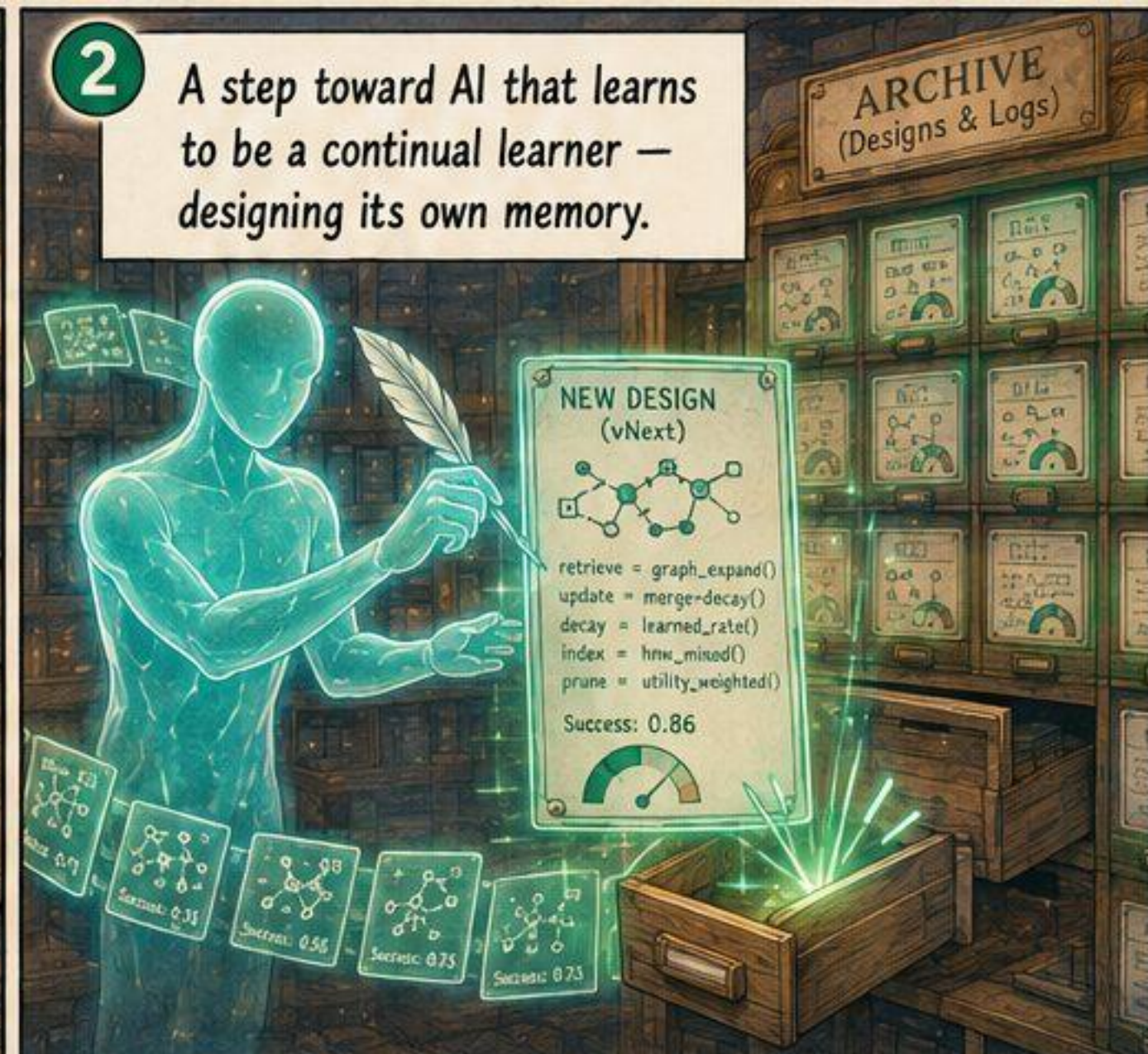
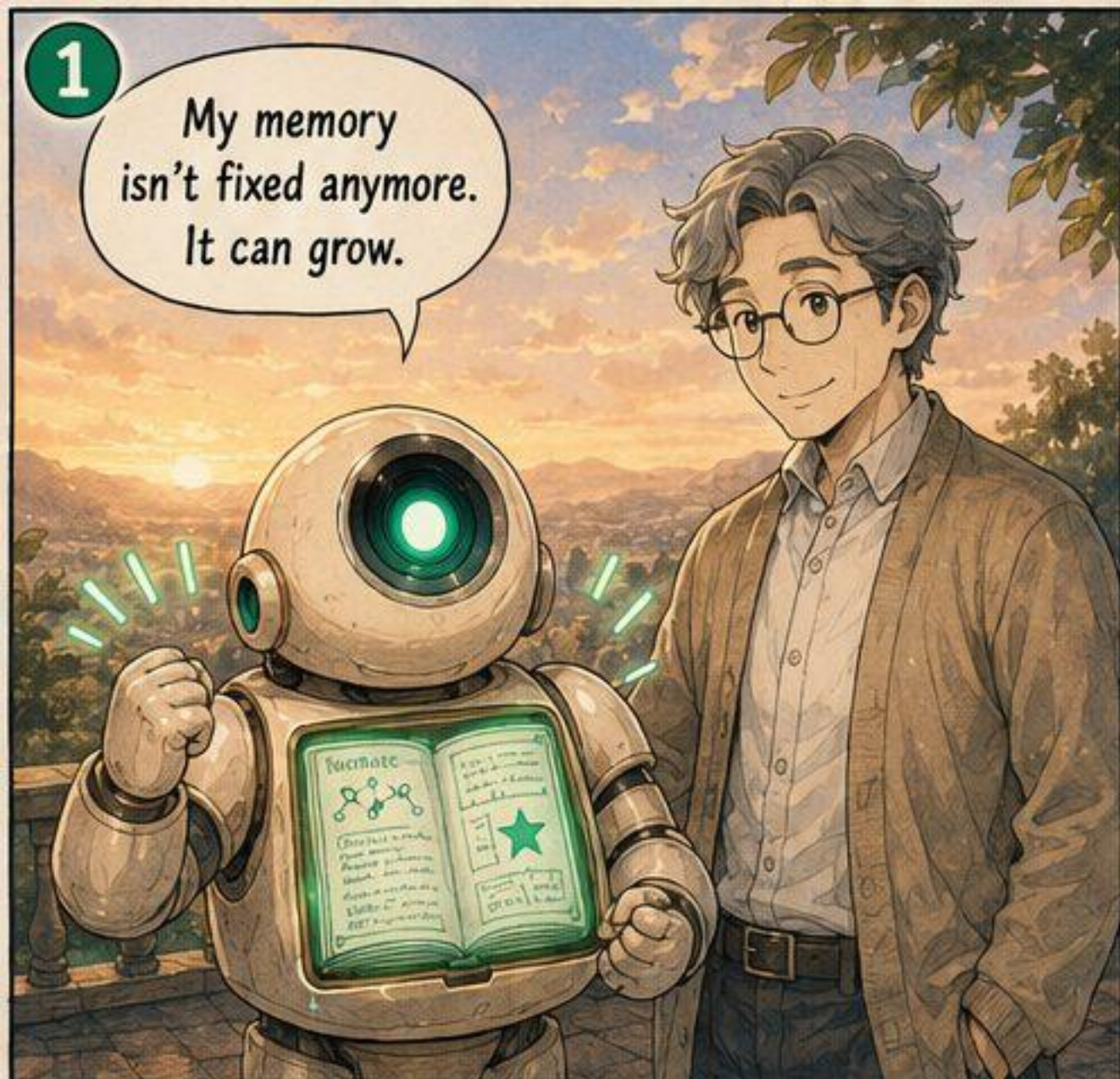
6 Better memory —
discovered, not
hand-drawn.

✧ THE RESULTS ✧

- Consistently better ✓
- Across 4 diverse worlds ✓
- More cost-efficient ✓

LEARNED TO REMEMBER,
REMEMBERED TO LEARN.







PANEL 1

PANEL 2

PANEL 3



PANEL 1

PANEL 2

PANEL 3



PANEL 1

PANEL 2

PANEL 3



PANEL 1

PANEL 2

PANEL 3



PANEL 1

PANEL 2

PANEL 3



PANEL 1

PANEL 2

PANEL 3



PANEL 1

PANEL 2

PANEL 3